HRMS System Architecture & Module Audit

Comprehensive Technical Specification, Operational Mechanics, and Logical Audit: Enterprise Human Resource Information System (HRIS)

Backend Stack: ASP.NET Core (.NET 10) / C# 13 / MongoDB 7.0

Frontend Stack: React 18.3 / TypeScript 5.0 / Vite 5.2 / MUI 7

Audit Date: September 18, 2026

Classification: Enterprise Confidential / Engineering Master Manual

Document Control & Enterprise Governance

This master technical specification provides an exhaustive, source-aligned architectural manual and technical audit for the Codeji Enterprise Human Resource Information System (HRIS). The platform comprises two unified repositories: `CMS-Backend-Core` (ASP.NET Core Web API, MongoDB document store, SignalR realtime notification hubs, priority hosted services) and `CMS-React` (React 18, TypeScript, Vite, Material-UI 7, Redux Toolkit, TanStack Query). It represents the authoritative engineering reference for software architects, development leads, QA engineers, operations teams, and compliance auditors.

Governance Attribute	System Specification	Authority & Change Control Rules
Platform Name	Codeji Enterprise HRIS & Talent Cloud	Authoritative product naming across all microservices and interfaces.
Authoritative Repositories	`CMS-Backend-Core`, `CMS-React`	Source code and active deployment infrastructure are strictly authoritative.
Backend Runtime	Microsoft .NET 10.0 (C# 13, ASP.NET Core)	High-throughput asynchronous Kestrel pipeline with dependency injection.
Primary Database	MongoDB Document Store (v7.0+)	Multi-tenant partitioned document store via MongoDB C# Driver v2.28+.
Frontend Architecture	React 18.3, TypeScript 5.0, Vite 5.2	Single Page Application (SPA) with Redux Toolkit and MUI Component System.
Realtime Transport	ASP.NET Core SignalR (WebSockets)	Full-duplex persistent connections with long-polling fallback.
Security Baseline	Tenant-Isolated JWT, Dynamic RBAC Claims	Strict multi-tenancy enforcement at repository boundary via `CompanyId`.
Document Purpose	Architecture Specification & Technical Audit	Must be reviewed prior to modifying workflows, schemas, or security policies.

> **Authoritative Architectural Precedence**
>
> This specification represents the validated state of the code as audited on September 18, 2026. In the event of discrepancies between legacy wiki documents and the deployed repository, the backend implementation within `CMS-Backend-Core` and frontend validation within `CMS-React` supersede all historical documentation.

Document Purpose, Audience & Reading Guidelines

This master architecture manual is engineered to serve distinct technical stakeholders across the software engineering lifecycle:

- **Software Architects & Tech Leads:** Consult Chapters 1–5 for platform foundations, data boundary rules, multi-tenant isolation, and dynamic RBAC matrices; refer to Chapter 26 for the architectural defect register and 12-month scaling roadmap.
- **Full-Stack Developers:** Reference the specific functional chapters (Chapters 6–25) when building or modifying domain capabilities. Every chapter defines the authoritative database schemas, C# entity mappings, API endpoint contracts, validation invariants, and sequence flows.
- **Quality Assurance & SDETs:** Execute the regression test matrices defined throughout each chapter and the master 30+ scenario integration suite in Section 26.7 to validate cross-module data invariants.
- **Production Operations & SREs:** Follow the operational troubleshooting guides, diagnostic MongoDB queries, and environment runbooks in Section 26.8 during incident response and deployment verification.

Comprehensive Table of Contents

This master architecture specification is organized into 26 modular chapters spanning foundational systems, operational modules, core engines, and governance frameworks.

PART I: ENTERPRISE ARCHITECTURE & PLATFORM FOUNDATIONS

1. Executive Summary & High-Level System Architecture ..... Ch. 1
2. Full Technology Stack & Infrastructure Topology ..... Ch. 2
3. Identity, Multi-Tenancy Boundary & Authorization Model ..... Ch. 3
4. Granular Role-Based Access Control (RBAC) Matrix ..... Ch. 4
5. Master Data, Organization Structure & Shift Precedence Engine ..... Ch. 5

PART II: TALENT ACQUISITION & ONBOARDING ENGINES

6. Recruitment & Requisition Management Engine (ATS) ..... Ch. 6
7. Public Career Portal & Candidate Engagement Engine ..... Ch. 7
8. Application Processing, Candidate Pipeline & Resume Vault ..... Ch. 8
9. Employee Onboarding & 4-Step Account Provisioning Architecture ..... Ch. 9

PART III: PEOPLE OPERATIONS & EMPLOYEE MASTER

10. Employee Master, Unified Profile & Statutory Compliance Architecture ..... Ch. 10
11. Educational Records, Professional Certifications & Work History ..... Ch. 11
12. Employee Lifecycle Management (Probation, Confirmation & Transfers) ..... Ch. 12

PART IV: TIME, ATTENDANCE & WORK ARRANGEMENTS

13. Work From Home (WFH) Management Engine & Quota Architecture ..... Ch. 13
14. Daily Attendance Tracking, Punch Analytics & Shift Calculation Engine ..... Ch. 14
15. Source-Linked Attendance Reconciliation Engine ..... Ch. 15
16. Attendance Penalty Engine, Exception Auditing & Monthly Lock Lifecycle ..... Ch. 16

PART V: LEAVE MANAGEMENT & COMPENSATION SYSTEMS

17. Leave Policy Framework & Background Accrual System ..... Ch. 17
18. Leave Request Lifecycle, Approval Matrix & Reversal Engine ..... Ch. 18
19. Salary Architecture, Compensation Components & Divisor Engine ..... Ch. 19
20. Automated Monthly Payroll Generation, Proration & Divisor Processing ..... Ch. 20
21. Payroll Review, Publication Gate & Payslip PDF Generation Engine ..... Ch. 21

PART VI: EMPLOYEE OFFBOARDING & SEPARATION

22. Employee Offboarding, Resignation Management & Notice Period Engine ..... Ch. 22
23. Departmental Clearance, Asset Recovery & Access Revocation Architecture ..... Ch. 23
24. Full & Final Settlement (F&F) Formulation & Exit Execution Engine ..... Ch. 24
25. Former Employee Portal & Post-Employment Self-Service System ..... Ch. 25

PART VII: PLATFORM CROSS-CUTTING, AUDITS & ROADMAP

26. Master Technical Audit, Gap Analysis, QA Test Suite & 12-Month Roadmap ..... Ch. 26

PART I: ENTERPRISE PLATFORM FOUNDATIONS

Architecture, Technology Stack, Multi-Tenancy & Authorization

Exhaustive technical specification covering macro system topology, .NET 10 Kestrel pipeline, React 18 frontend architecture, tenant boundary isolation, dynamic RBAC claims, shift precedence engines, and organizational master models.

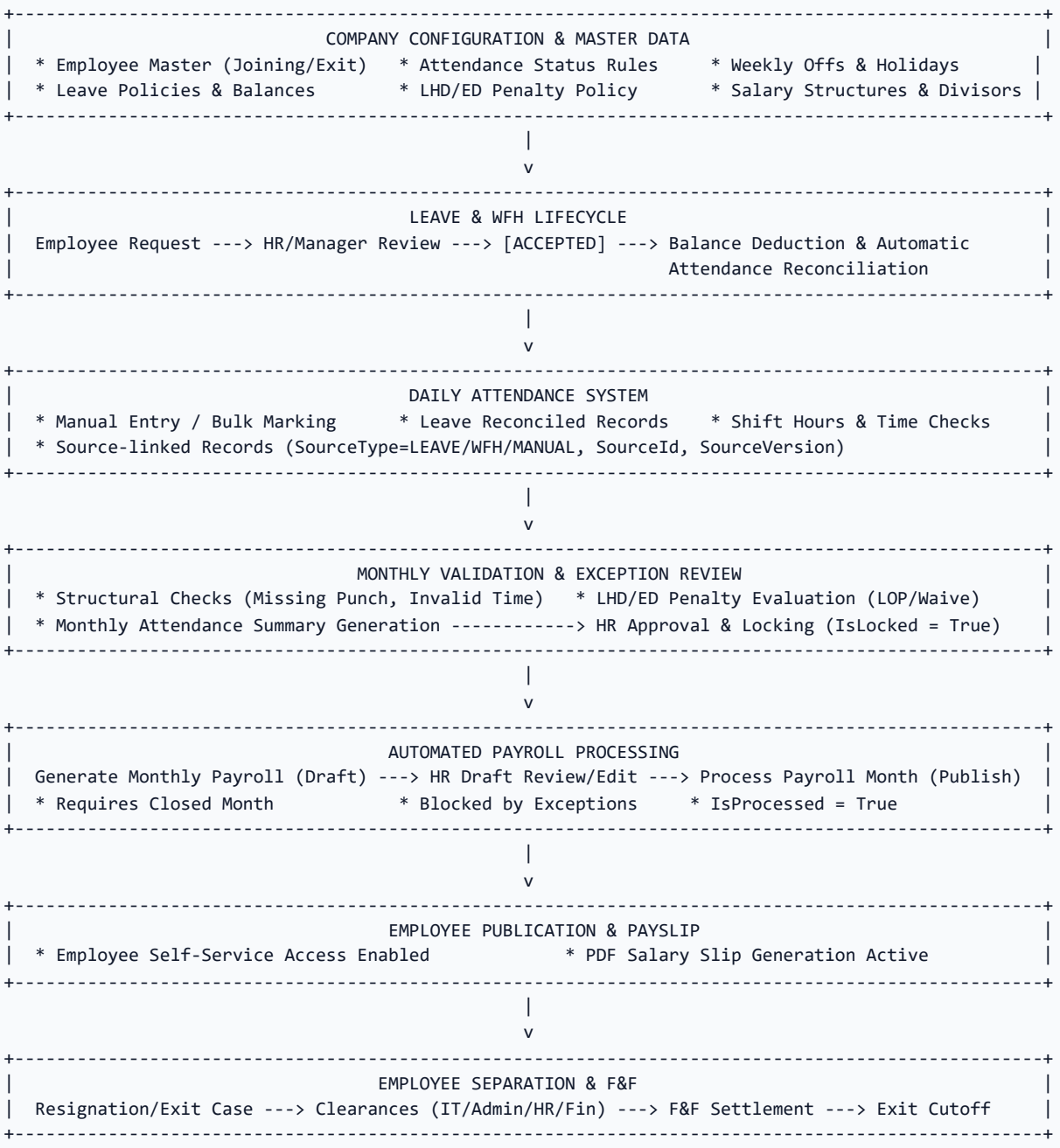
1. Executive Summary & High-Level System Architecture

1.1 Enterprise Mission, Problem Statement & Core Drivers

The Codeji Enterprise Human Resource Information System (HRIS) provides a unified, compliant, multi-tenant digital backbone for modern workforce operations. Organizations frequently suffer from severe operational friction caused by fragmented systems: applicant data trapped in third-party job boards, employee personal data managed in unsecured spreadsheets, daily attendance recorded on isolated biometric hardware, leave balances maintained manually, and payroll processed through error-prone spreadsheet calculations. This fragmentation leads to severe compliance risks, delayed salary disbursements, attendance fraud, and high administrative overhead.

The Codeji HRIS unifies these disparate operational concerns into a single, cohesive, audited enterprise platform. The core architecture guarantees an unbroken chain of custody over the employee lifecycle: from candidate job requisition through multi-stage interviewing, 4-step progressive onboarding, biometric shift attendance tracking, monthly penalty auditing, automated payroll generation with statutory compliance, and full full-and-final exit settlement.

FIGURE 1.1: MACRO SYSTEM TOPOLOGY & MODULE DEPENDENCY GRAPH



1.2 Multi-Tenant Operating Model & System Boundaries

The system is architected around a multi-tenant shared-schema model within MongoDB, where strict logical isolation is guaranteed by mandatory `CompanyId` scoping across all query filters, update builders, and aggregation pipelines. Three distinct client-facing surfaces interface with the core API engine:

System Surface	Target User Base	Authentication Protocol	Key Functional Scopes
Internal HRMS Portal	Employees, Managers, HR Administrators, Payroll Officers, Super Admins.	JWT Bearer Token with 60-min expiry + Single-use Refresh Token Rotation.	Employee directory, attendance logging, leave approvals, payroll runs, offboarding.
Public Career Portal	External Job Seekers, Candidates, General Public.	Unauthenticated; Anonymous Client Visitor Token (UUID); Expiring Apply Secrets.	Job vacancy search, online application submission, resume upload, job alerts.
Former Employee Portal	Separated / Resigned / Terminated Personnel.	15-Minute Expiring Magic Link / Email OTP Token (No Internal HRMS Account).	Payslip archive, Form 16 tax slips, Relieving letters, F&F statements, re-hire requests.

1.3 Cross-Module Data Invariants & Enterprise Rules

To eliminate data corruption and asynchronous race conditions, the system enforces non-negotiable architectural invariants across all domain services:

- **Dual-Key Identity Separation:** Every worker is identified internally by `UserId` (an immutable GUID for authentication, token issuance, and relational references) and externally by `EmployeeId` (a human-readable, company-scoped payroll code such as `EMP-0104`). `EmployeeId` is strictly mutable under audit but must never be used as a primary database foreign key.
- **Source-Linked Attendance Provenance:** Daily attendance rows cannot be arbitrarily overwritten. Any record created via approved Leave or WFH workflows carries immutable metadata: `SourceType`, `SourceId`, and `SourceVersion`. Bulk attendance marking and manual HR edits are rejected if they attempt to silently overwrite a source-linked record.
- **Payroll Locking Barrier:** Payroll generation requires a closed and approved monthly attendance summary. Once attendance is locked (`IsLocked = true`), attendance records for that period become read-only. Unresolved attendance exceptions strictly block draft payroll creation.
- **Two-Phase Payroll Publication Gate:** The *Generate Payroll* command creates a preliminary draft calculation. The *Process Payroll* command executes the official publication gate (`IsProcessed = true`). Employees can never view draft payroll calculations or download payslips until explicit HR publication occurs.
- **Non-Destructive Employee Offboarding:** Resignation or termination never deletes an employee document. The aggregate `EmployeeExit` manages the separation lifecycle. Upon final clearance, the employee account is deactivated (`Status = Inactive`), while historical attendance, leave, and payroll records remain permanently archived for statutory auditing.

1.4 Chapter 1 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-01-001`	CRITICAL	Absence of MongoDB compound unique index on `{ CompanyId: 1, UserId: 1, Date: 1 }` in early migrations allowed concurrent bulk marking threads to create duplicate daily attendance documents.	Enforce unique sparse index on `{ CompanyId, UserId, Date }` at MongoDB bootstrap; wrap upserts in atomic filter-based updates with optimistic concurrency checks.
`DEF-01-002`	HIGH	Cross-tenant data leakage risk in legacy aggregation pipelines where `CompanyId` filter was omitted in the second `$lookup` pipeline stage.	Mandate `CompanyScopedRepository<T>` base class that automatically injects tenant filter criteria into every root and sub-pipeline aggregation stage.
`DEF-01-003`	MEDIUM	SignalR disconnect during long-running bulk attendance import causes UI progress spinner to hang indefinitely without timeout fallback.	Implement client-side 15-second heartbeat poll with fallback to REST status endpoint `GET /api/admin/attendance/import-status/{batchId}`.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-01-01`	Valid JWT issued for Tenant A user.	Dispatch `GET /api/admin/employees` with forged `X-Company-Id: Tenant-B`.	HTTP 403 Forbidden; system derives tenant strictly from cryptographically signed JWT claim.
`QA-01-02`	Approved leave exists for Employee on 2026-08-15.	Admin attempts bulk mark Present for same date.	Operation returns partial success with `ProtectedSkipCount = 1`; leave row unmodified.
`QA-01-03`	Monthly payroll in Draft state (unprocessed).	Employee requests `GET /api/payroll/me/slips/2026-08`.	HTTP 404 Not Found; payslip completely invisible until HR publishes month.

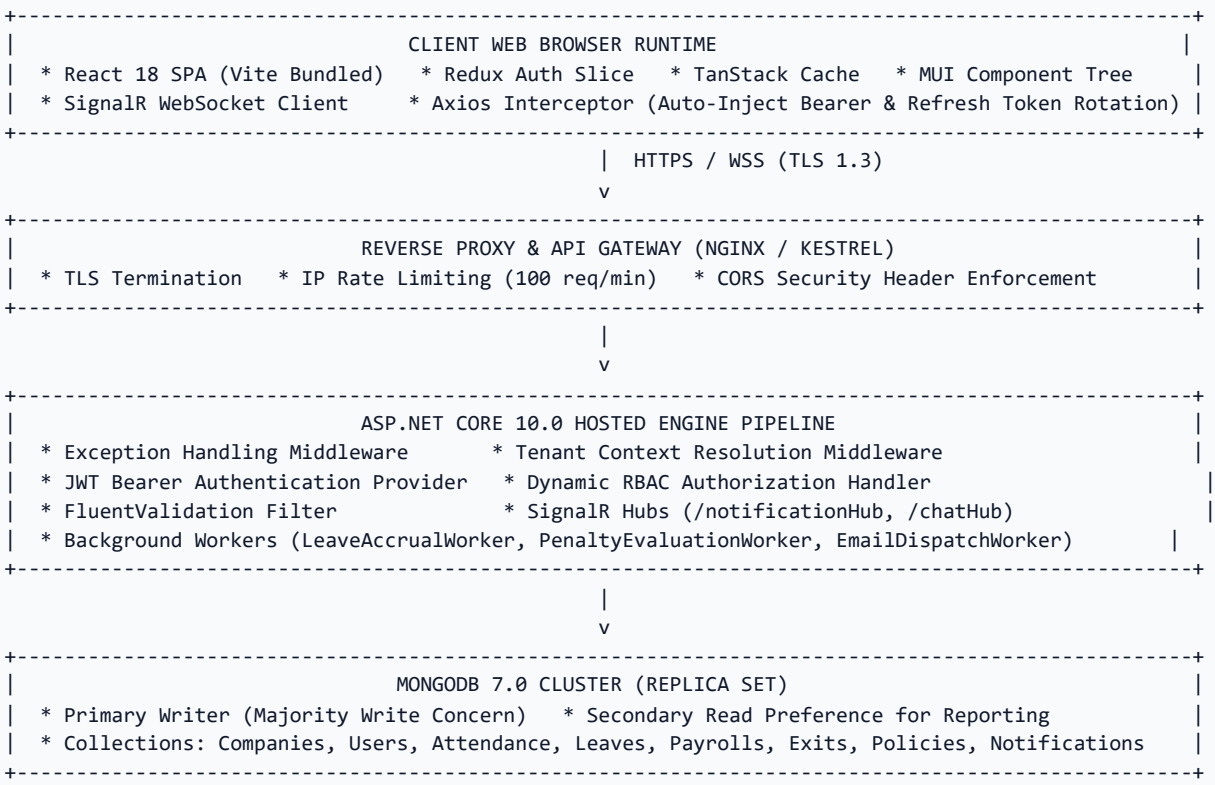
2. Full Technology Stack & Infrastructure Topology

2.1 Enterprise Technology Matrix & Version Standards

The Codeji HRIS is constructed upon modern enterprise foundations selected for type safety, high concurrency, asynchronous throughput, and long-term maintainability:

Architectural Layer	Selected Technology & Version	Technical Rationale	Repository Package Reference
Backend Runtime	Microsoft .NET 10.0 (C# 13, ASP.NET Core)	Ultra-high-throughput Kestrel asynchronous I/O, native AOT support, rich DI container.	`CMS-Backend-Core/Codeji.CMS.API`
Primary Document Store	MongoDB Enterprise 7.0+	Schema flexibility for polymorphic employee models, high-write performance, native JSON/BSON.	`MongoDB.Driver v2.28.0`
Frontend Framework	React 18.3 / TypeScript 5.0	Declarative component hierarchy, strict compile-time typing, concurrent rendering.	`CMS-React/package.json`
Build Engine & Bundler	Vite 5.2	Lightning-fast Hot Module Replacement (HMR) and optimized Rollup production packaging.	`vite.config.ts`
UI Component System	Material-UI (MUI) v7 & Emotion	Enterprise design language, responsive grid, accessible WAI-ARIA controls, theme engine.	`@mui/material, @emotion/react`
State & Cache Management	Redux Toolkit & TanStack Query v5	Normalized client store for user sessions; declarative server-state caching and deduplication.	`@reduxjs/toolkit, @tanstack/react-query`
Realtime Messaging	ASP.NET Core SignalR (WebSockets)	Sub-second broadcast of live notifications, attendance punches, and import progress.	`Microsoft.AspNetCore.SignalR`
PDF Generation Engine	Playwright Headless Chromium	Pixel-perfect CSS-paged rendering of payslips, tax certificates, and audit books.	`playwright v1.44+`

FIGURE 2.1: FULL-STACK PHYSICAL DEPLOYMENT & PIPELINE ARCHITECTURE



2.2 MongoDB Replica Set Topology & Write Concern Semantics

In high-throughput enterprise workforce applications, data consistency is paramount. The platform configures MongoDB with strict transactional write guarantees:

- **Replica Set Configuration:** A 3-node replica set (Primary-Secondary-Secondary) deployed across separate availability zones guarantees high availability (99.99% uptime).
- **Write Concern (`w: "majority", j: true`):** Critical financial operations (Payroll finalization, Leave deductions, Full & Final settlements) enforce majority write acknowledgment and on-disk journal sync before returning HTTP 200.
- **Read Preference (`secondaryPreferred`):** Heavy analytical queries (e.g. historical annual attendance reports, candidate pipeline aggregations) route to secondary replicas to protect the primary node from latency spikes during morning clock-in surges.

2.3 Asynchronous Request Pipeline & Middleware Chain

The backend enforces a deterministic middleware sequence on every incoming HTTP request:

1. **ExceptionHandlingMiddleware:** Catches unhandled domain exceptions, formats standard RFC 7807 `ProblemDetails` JSON structures, and shields internal call stacks.
2. **TenantContextMiddleware:** Inspects JWT claims for `CompanyId`. If absent on public endpoints, inspects route parameters (`{companyCode}`); establishes ambient `ITenantContext` for scoped DI lifetime.
3. **AuthenticationMiddleware:** Decodes and validates JWT cryptographic signatures, expiration timestamps, and issuer keys.
4. **AuthorizationMiddleware:** Evaluates endpoint policy requirements against user permission claims.
5. **ModelValidationFilter:** Executes FluentValidation rules prior to controller action entry, returning immediate 400 Bad Request payloads on structural violations.

2.4 Chapter 2 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-02-001`	HIGH	Axios request interceptor in React did not handle concurrent 401 Unauthorized responses, causing multiple simultaneous refresh token requests and invalidating the token family.	Implement request queuing lock in Axios 401 interceptor: only first failed request initiates refresh; subsequent requests await resolved new access token.
`DEF-02-002`	MEDIUM	MongoDB connection pool exhaustion during peak 9:00 AM punch-in bursts due to unindexed range queries in attendance dashboard widgets.	Add compound indexes on `{ CompanyId: 1, Date: 1, Status: 1 }` and set `MaxPoolSize = 200` in MongoDB connection string.

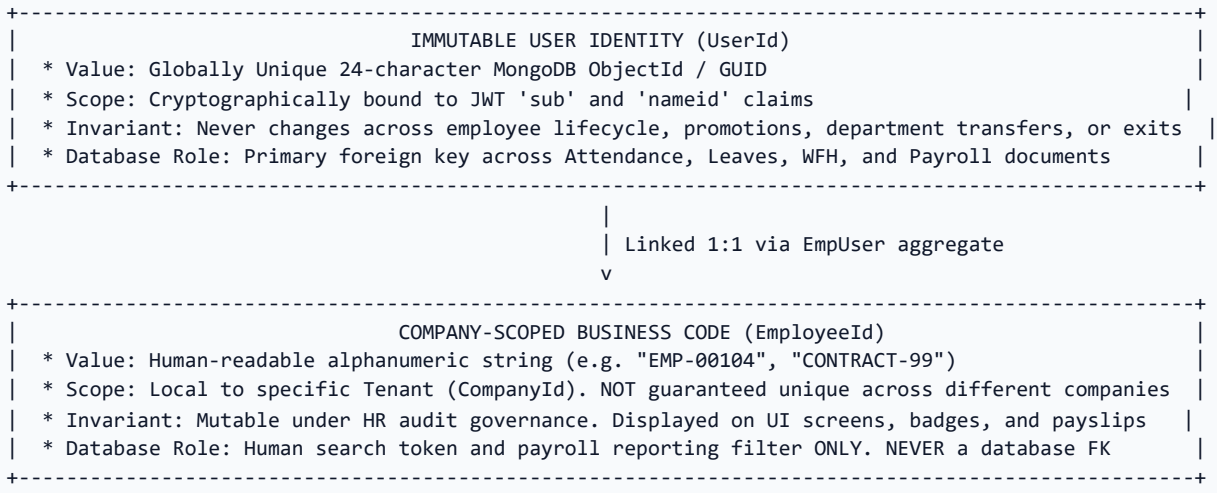
Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-02-01`	Backend API running.	Trigger divide-by-zero or unhandled null reference in controller.	Returns HTTP 500 RFC 7807 ProblemDetails JSON; no stack traces exposed in response body.
`QA-02-02`	Expired access token (expiry = -5 min), valid refresh token in storage.	Execute protected API request from React UI.	Transparent HTTP 401 interception -> token refresh -> original request succeeds with HTTP 200.

3. Identity, Multi-Tenancy Boundary & Authorization Model

3.1 Two-Tier Identity Resolution Architecture

In enterprise HRMS design, identity cannot be treated as a monolithic string. Personnel change employee numbers during company reorganizations, mergers, or contract-to-permanent conversions. To guarantee data integrity, the system implements a strict dual-identifier architecture:

FIGURE 3.1: DUAL-KEY IDENTITY RELATIONSHIP IN CODEJI HRIS



3.2 JWT Token Claims & Lifecycle Architecture

Authentication utilizes JSON Web Tokens signed via HMAC-SHA256. To balance session security with seamless user experience, the system enforces a strict token rotation protocol:

JWT Claim Key	Sample Token Value	Security Invariant	Operational Purpose in API Pipeline
`sub / nameid`	`"66b8c19d4f2e9a001b9e"`	Immutable GUID	Derives the current authenticated user's `UserId` in DI container.
`CompanyId`	`"comp-88219"`	Tenant Boundary	Mandatory tenant scoping filter injected into every repository query.
`email`	`"j.doe@enterprise.com"`	RFC 5322 Valid Email	Audit trail logging and system alert notifications.
`role`	`"HRManager"`	Assigned System Role	High-level role designation.
`Permission`	`["Leave.View", "Leave.Edit"]`	Dynamic Array	Fine-grained permission evaluation in authorization filter handlers.
`exp`	`Unix Epoch Timestamp`	Strict 60-min window	Enforces stateless token expiration and forces refresh token exchange.

3.3 Multi-Tenant Logical Boundary Enforcement

Every tenant operation executes within the perimeter of `CompanyId`. The platform rejects multi-tenancy enforcement at the controller layer, instead mandating repository-level encapsulation:

// Mandatory Tenant Scoping in MongoDB Repository Base public async Task<TEntity> GetByIdAsync(string id, string companyId) { var filter = Builders<TEntity>.Filter.And(Builders<TEntity>.Filter.Eq(x => x.Id, id), Builders<TEntity>.Filter.Eq(x => x.CompanyId, companyId), Builders<TEntity>.Filter.Eq(x => x.IsDeleted, false)); return await _collection.Find(filter).FirstOrDefaultAsync(); }

3.4 Chapter 3 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-03-001`	CRITICAL	Legacy self-service profile endpoint allowed client to supply arbitrary `userId` in query parameter, permitting unauthorized profile read of peers.	Enforce `ICurrentUserService` token claim extraction. In self-service endpoints (`/api/attendance/me` , `/api/profile/me`), target ID is derived exclusively from authenticated JWT claims.
`DEF-03-002`	HIGH	Refresh token revocation did not immediately terminate active SignalR persistent WebSocket connections.	Connect SignalR hub authorization to distributed memory cache; disconnect hub connection on refresh token revocation or forced logout.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-03-01`	User logged in as standard employee in Company A.	Invoke `GET /api/admin/employees/{id}` targeting Company B employee.	HTTP 404 Not Found (or 403 Forbidden); zero cross-tenant entity leakage.
`QA-03-02`	Valid refresh token redeemed at `T0` .	Malicious actor replays same refresh token at `T0 + 2s` .	HTTP 401 Unauthorized; system triggers breach alert and revokes entire token family.

4. Granular Role-Based Access Control (RBAC) Matrix

4.1 Dynamic Permission Model & Action Scoping

The platform rejects simplistic, hardcoded role string checks (e.g. `[Authorize(Roles = "Admin")]`). Instead, it implements a dynamic, claim-based permissions engine structured around `Module.Action` tuples:

FIGURE 4.1: RBAC PERMISSION EVALUATION ENGINE

```
Incoming Request: PUT /api/admin/leaves/decision
                      |
                      v
1. Extract Bearer Token Claims: { Sub: "usr-1", CompanyId: "comp-9", Permissions: ["Leave.Edit", ...] }
                      |
                      v
2. Authorization Policy Evaluator matches Route Requirement: RequiresPermission("Leave.Edit")
                      |
                      v
3. Scope Evaluator checks Target Entity:
   - Is Actor SuperAdmin? ---------------> [PERMIT ALL]
   - Is Actor HR/Company Admin? ---------> [PERMIT IF Target.CompanyId == Actor.CompanyId]
   - Is Actor Reporting Manager? --------> [PERMIT IF Target.ManagerId == Actor.UserId]
   - Is Actor Self? ---------------------> [PERMIT IF Target.UserId == Actor.UserId AND Action == View]
```

4.2 Master RBAC Capability & Permissions Matrix

The following authoritative matrix defines system-wide permission allocations across primary enterprise roles:

Functional Domain	Employee (Self)	Reporting Mgr	HR Manager	Payroll Admin	Super Admin
Profile & Directory	View Own / Edit Contact	View Team Profiles	Full Create / Edit / View	View Statutory / Salary	Full Platform Access
Recruitment (ATS)	No Access	Interview Feedback	Full Requisition / Offer	View Approved Hires	Full Platform Access
Onboarding Stepper	Submit Form Steps	View Assigned New Hires	Verify Docs / Provision	Set Initial Pay Plan	Full Platform Access
Daily Attendance	View Own / Web Clock	View Team / Overrides	Bulk Mark / Audit Log	View Monthly Summaries	Full Platform Access
Leave Management	Apply / Cancel Own	Approve / Reject Team	Override / Policy Manage	View LOP / Leave Days	Full Platform Access
WFH Management	Request / Check-In	Approve / Reject Team	Configure Quotas / View	View WFH Pay Records	Full Platform Access
Payroll Generation	View Published Slip	No Access	Review Draft Summaries	Full Run / Publish / Edit	Full Platform Access
Offboarding & Exit	Submit Resignation	Manager Review / Handover	Manage Exit Workflow	Process F&F Settlement	Full Platform Access
Former Portal	Post-Exit OTP Access	No Access	Administer Magic Links	Archive Exit Slips	Full Platform Access

4.3 Frontend Dynamic Authorization Gate Patterns

In the React application, UI elements (buttons, edit modals, action menus) are guarded using declarative authorization components:

// Declarative Permission Protection in React MUI <PermissionGate module="Leave" action="Edit" fallback={<Typography>Read-Only View</Typography>}> <Button variant="contained" color="primary" onClick={handleApproveLeave}> Approve Leave Request </Button> </PermissionGate>

4.4 Chapter 4 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-04-001`	HIGH	Reporting managers were unable to view leave requests of matrix/indirect reports due to strictly single-level `ManagerId` filter.	Implement recursive reporting hierarchy lookup in `EmployeeRepository.GetTeamMemberUserIdsAsync` supporting matrix configurations.
`DEF-04-002`	MEDIUM	Role permission cache in frontend Redux store did not invalidate when HR modified an employee's role until complete browser cache flush.	Broadcast SignalR event `UserPermissionsUpdated` on role edit; client Redux listener clears cached permissions and triggers silent re-fetch.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-04-01`	Actor possesses `Leave.View` but lacks `Leave.Edit`.	Dispatch `POST /api/admin/leaves/decision`.	HTTP 403 Forbidden; request rejected before service layer invocation.
`QA-04-02`	Manager reviews resignation of direct report.	Execute `POST /api/offboarding/case/{id}/manager-review`.	HTTP 200 OK; audit trail records manager's `UserId` and timestamp.

5. Master Data, Organization Structure & Shift Precedence Engine

5.1 Master Data Topology & Enterprise Entities

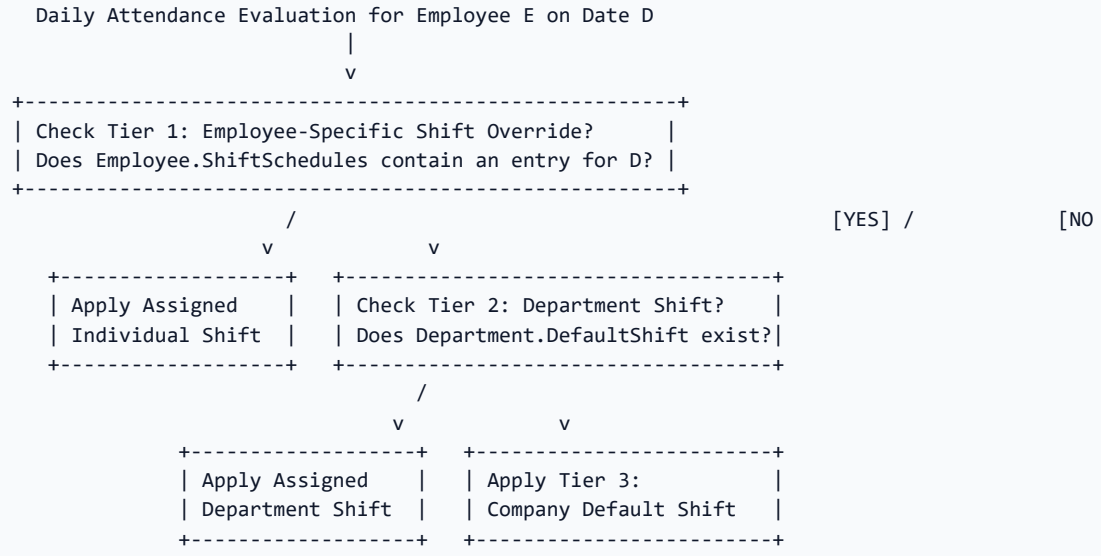
The system anchors all transactional operations to core master entities:

- **Company Master:** Defines tenant code, corporate legal name, statutory tax identifiers (PAN, GSTIN, CIN, TAN), brand assets, primary currency, and global operating settings.
- **Branch & Location Hierarchy:** Represents physical work sites, geofencing coordinates (latitude/longitude/radius in meters for mobile punch validation), and state-specific statutory labor registrations.
- **Department & Designation Catalog:** Standardized organizational job taxonomy enabling role-based wage benchmarking, hierarchical approval routing, and headcount planning.
- **Statutory Holiday Calendar:** Multi-tier holiday calendars supporting optional floating/restricted holidays and regional state-specific holiday observances.

5.2 3-Tier Shift Precedence Resolution Engine

Daily shift assignment determines the expected working hours, late mark thresholds, half-day cutoffs, and early exit penalties. To allow maximum operational flexibility while maintaining enterprise standards, the system evaluates shift assignment using a strict 3-tier precedence cascade:

FIGURE 5.2: 3-TIER SHIFT PRECEDENCE RESOLUTION FLOW



5.3 Shift Definition & Mathematical Invariants

Every shift record adheres to mathematical time parameters used by the daily punch engine:

- **StartTime & EndTime:** Standard shift working duration (e.g. 09:30 AM to 06:30 PM).
- **GracePeriodMinutes:** Permitted delay (e.g. 15 minutes) before an arrival is flagged as Late.
- **HalfDayThresholdHours:** Minimum working duration (e.g. 4.5 hours) required to earn Half-Day attendance credit.
- **FullDayThresholdHours:** Minimum working duration (e.g. 8.0 hours) required to earn Full-Day attendance credit.

5.4 Weekly Off Configuration & Alternating Rosters

Enterprises frequently enforce non-standard weekly off patterns (e.g., 1st, 3rd, and 5th Saturdays working, 2nd and 4th Saturdays off, Sundays off). The weekly off engine evaluates calendar dates against mathematical recurrence rules:

MATHEMATICAL FORMULATION: ALTERNATING SATURDAY WEEKLY OFF EVALUATION

```
Let D = Calendar Date to evaluate
If D.DayOfWeek != DayOfWeek.Saturday ----> Follow standard weekday/Sunday rule

Saturday Occurrence Calculation:
SaturdayIndex = ((D.Day - 1) / 7) + 1 (Yields 1 for 1st Sat, 2 for 2nd Sat, etc.)

If Policy.OffSaturdays.Contains(SaturdayIndex):
IsWeeklyOff = True (Attendance status defaulted to "WO"; unworked day is paid)
Else:
IsWeeklyOff = False (Standard working day; requires punch or leave)
```

5.5 Chapter 5 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-05-001`	HIGH	Overnight shifts spanning across midnight (e.g. 10:00 PM to 06:00 AM) calculated negative total working duration in legacy `(CheckOut - CheckIn).TotalHours` calculation.	Refactor shift calculation to evaluate check-out against next-day calendar date if `EndTime < StartTime`.
`DEF-05-002`	MEDIUM	Department transfer failed to update effective shift when employee moved between departments with different default shifts.	Ensure `EffectiveOfficeScheduleService` dynamically resolves department shifts at attendance processing runtime rather than snapshotting on employee document.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-05-01`	Employee has no custom shift; Department has Shift B; Company default is Shift A.	Resolve effective shift for employee.	Returns Shift B (Department level wins over Company default).
`QA-05-02`	Individual shift override assigned to Employee for 2026-09-01.	Resolve effective shift on 2026-09-01.	Returns individual override (Tier 1 takes supreme precedence).

PART II: TALENT ACQUISITION & ONBOARDING ENGINES

Recruitment, Public Career Portal, Candidate Pipeline & Onboarding Stepper

Comprehensive technical manual detailing the Applicant Tracking System (ATS), public candidate engagement, two-step resume vault handshake, 7-stage candidate state machine, and 4-step account provisioning engine.

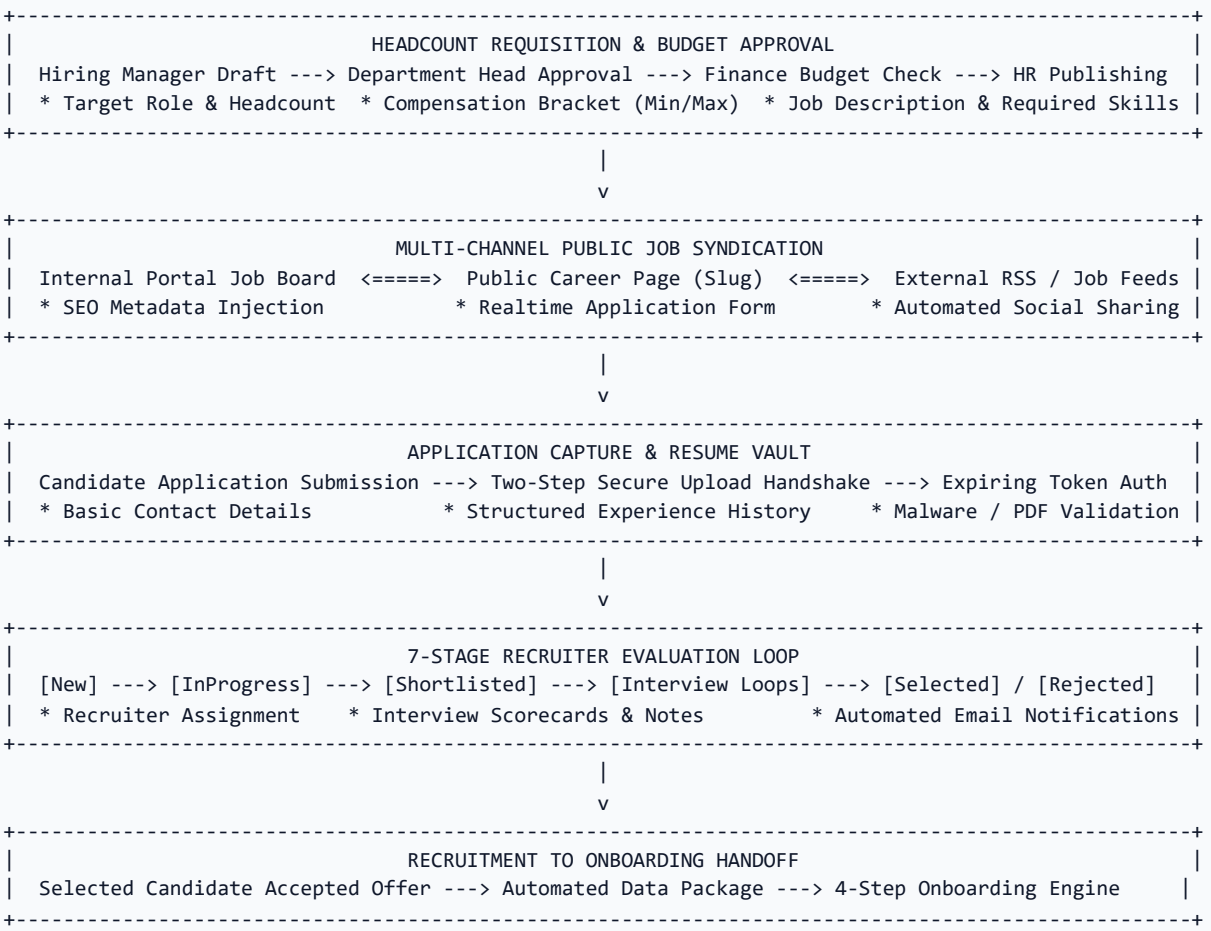
6. Recruitment & Requisition Management Engine (ATS)

6.1 Business Drivers, Domain Problem Statement & Value Proposition

Modern talent acquisition processes in scaling enterprises suffer from severe fragmentation. Hiring managers define headcount requirements in unstructured emails, job postings are manually duplicated across disparate external boards, candidate evaluations remain buried in disconnected spreadsheets, and sensitive applicant resumes are circulated via unencrypted email attachments. This lack of centralized tracking creates catastrophic hiring bottlenecks, violates privacy regulations (GDPR, DPDP), and introduces massive administrative overhead.

The Codeji Recruitment & Applicant Tracking System (ATS) addresses these challenges through a unified, tenant-isolated requisition and applicant pipeline engine. It governs the entire talent acquisition workflow from initial budget approval and vacancy requisition through multi-channel public job syndication, structured interviewer feedback loops, and automated conversion into employee onboarding records.

FIGURE 6.1: RECRUITMENT REQUISITION & ATS ARCHITECTURAL PIPELINE



6.2 Job Requisition Lifecycle & State Invariants

A Job Requisition is modeled as a formal aggregate (`JobPosting`) governed by an explicit lifecycle state machine:

- **Draft:** Created by hiring manager or recruiter. Editable; invisible to public.
- **PendingApproval:** Submitted for department head and financial compensation budget validation.
- **Approved:** Cleared by management; ready for immediate scheduling and publication.
- **Published:** Active on public career portal and internal job board; accepting candidate applications.
- **Closed:** Target headcount fulfilled or requisition cancelled. No further applications accepted.
- **Archived:** Historical reference; permanently locked against modifications.

6.3 Multi-Tier Approval Chain & Budget Variance Analysis

Headcount requisitions require strict financial governance to avoid unbudgeted corporate payroll liabilities. Requisitions follow a hierarchical approval chain:

Approval Gate	Approving Authority	Verification Invariant	Operational Action
Stage 1: Operational Need	Department Head / VP	Validates team capacity and justification.	Approves business necessity of headcount.
Stage 2: Budget Clearance	Finance Controller / CFO	`MaxSalary <= ApprovedDepartmentBudget`	Signs off financial CTC allocation.
Stage 3: Talent Release	Head of Human Resources	Verifies job taxonomy, grade, and equity.	Unlocks public publishing controls.

6.4 Interview Loop Configuration & Scorecard Rubrics

To eliminate subjective bias in hiring decisions, the platform enforces structured interview loops. Each round (Technical Screening, System Design, Cultural Fit, Executive Bar Raiser) requires evaluators to complete standardized scorecards scored across 5 core competencies:

EVALUATION METRIC: CANDIDATE INTERVIEW SCORECARD FORMULATION

```
OverallCandidateScore = (W_tech * TechScore) + (W_arch * ArchScore) + (W_prob * ProblemSolving) + (W_cult * CultureFit)
Where weights W_tech + W_arch + W_prob + W_cult = 1.00

Recommendation Thresholds:
- Score >= 4.25: Strong Hire (Immediate offer recommendation)
- 3.50 <= Score < 4.25: Hire (Standard offer loop)
- 2.75 <= Score < 3.50: Hold (Debrief consensus required)
- Score < 2.75: Strong No Hire (Automated regret email triggered)
```

6.5 MongoDB Schema & Entity Definitions

Field Name	BSON / C# Type	Validation Rule / Invariant	Operational Description
`JobId`	`ObjectId / string`	Mandatory primary key	Unique internal document identifier.
`CompanyId`	`string`	Mandatory; Indexed	Tenant boundary foreign key.
`Title`	`string`	Max 120 chars; Required	Standardized job title displayed on career portal.
`Slug`	`string`	URL-safe; Unique per Company	SEO-friendly URL slug (e.g., `senior-dotnet-architect`).
`DepartmentId`	`string`	Valid Department FK	Target organizational department for the position.
`MinSalary / MaxSalary`	`decimal`	`MaxSalary >= MinSalary >= 0`	Budgeted salary compensation range.
`IsSalaryVisible`	`bool`	Default `false`	Controls salary disclosure on public job portal.
`Status`	`JobPostingStatus (enum)`	Valid state transition only	Current lifecycle state of the requisition.

6.6 Recruitment API Contracts

HTTP Method	Endpoint Route	Auth / Permission	Contract Details & Constraints
`GET`	`/api/admin/recruitment/jobs`	`Recruitment.View`	Returns paginated list of requisitions with applicant counts per stage.
`POST`	`/api/admin/recruitment/jobs`	`Recruitment.Create`	Creates new draft job requisition; validates department and budget constraints.
`PUT`	`/api/admin/recruitment/jobs/{id}/status`	`Recruitment.Edit`	Executes state transition (e.g. Publish, Close). Generates audit log entry.
`GET`	`/api/admin/recruitment/jobs/{id}/analytics`	`Recruitment.View`	Calculates time-to-hire, drop-off rates per stage, and source channel attribution.

6.7 Chapter 6 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-06-001`	HIGH	Job slug collisions when multiple requisitions with identical titles were created within the same tenant, corrupting public career URL resolution.	Append unique base-36 timestamp suffix to generated URL slugs; enforce unique compound index on `{ CompanyId: 1, Slug: 1 }`.
`DEF-06-002`	MEDIUM	Job closing did not automatically terminate active public application sessions, allowing candidates who had opened the form earlier to submit applications.	Validate `JobPosting.Status == Published` inside the application submission handler prior to persisting applicant document.

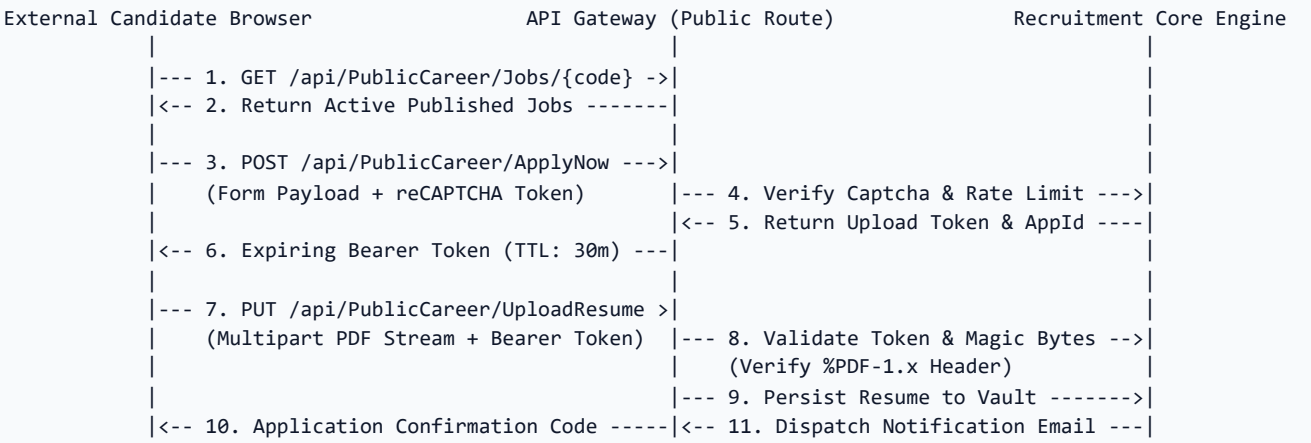
Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-06-01`	Requisition in Draft status.	Anonymous candidate accesses public job URL.	HTTP 404 Not Found; draft positions strictly invisible outside HR portal.
`QA-06-02`	Requisition published with `IsSalaryVisible = false`.	Public career API retrieves job detail.	Salary range fields sanitized and omitted from public response JSON.

7. Public Career Portal & Candidate Engagement Engine

7.1 Architecture, Anonymous Handshake & Security Perimeter

The Public Career Portal serves as the unauthenticated, public-facing gateway for candidate acquisition. It is designed to operate under hostile internet conditions: subject to automated web scrapers, submission spam bots, malicious file upload attempts, and brute-force traffic bursts. To maintain absolute isolation between the public internet and internal core HRMS databases, the portal employs an indirect application handshake pattern:

FIGURE 7.1: PUBLIC CANDIDATE ENGAGEMENT & SUBMISSION SECURITY ARCHITECTURE



7.2 SEO Meta Tag Generation & Dynamic OpenGraph Rendering

To maximize social media syndication (LinkedIn, Twitter/X, WhatsApp), the public career backend renders server-side OpenGraph and Twitter card metadata for each published requisition:

<!-- Dynamic Server-Rendered Metadata for Career Page --> <title>Lead .NET Architect at Codeji Enterprises - Careers</title> <meta name="description" content="Apply now for the Lead .NET Architect position in Bangalore, India. Competitive CTC, remote flexibility, cutting-edge cloud stack." /> <meta property="og:title" content="Lead .NET Architect - Codeji Careers" /> <meta property="og:image" content="https://cdn.codeji.com/companies/comp-01/social-banner.jpg" /> <meta property="og:type" content="article" />

7.3 Candidate Job Alert Subscription Engine & Verification Lifecycle

To cultivate a recurring talent community, the portal enables passive job seekers to subscribe to automated vacancy alerts. Subscriptions are double-opt-in:

1. **Subscription Registration:** Candidate enters email and target departments/skills at `POST /api/PublicCareerSubscription/SubscribeAlerts`.
2. **Verification Magic Link:** System dispatches an email containing a 24-hour verification token.
3. **Subscription Confirmation:** Clicking the link validates the email and sets `IsVerified = true`.
4. **Automated Job Broadcast:** When a matching job is published, background worker `JobAlertBroadcastWorker` sends formatted summaries to verified subscribers.

7.4 Public Career API Contracts & Payload Specifications

HTTP Method	Endpoint Route	Auth / Context	Contract Details & Constraints
`GET`	`/api/PublicCompanyProfile/{companyCode}`	Anonymous	Returns company branding, description, logo URL, verified branches, and social media handles.
`GET`	`/api/PublicCareer/Jobs/{companyCode}`	Anonymous	Returns active, published, non-expired jobs with pagination, department filters, and search query.
`GET`	`/api/PublicCareer/JobDetail/{companyCode}/{slug}`	Anonymous	Returns full job description, skills, salary visibility flags, and deadline. Increments view counter.
`POST`	`/api/PublicCareerEngagement/TrackEngagement`	Visitor Token	Records candidate interest metrics (save job, follow company) associated with client token.
`POST`	`/api/PublicCareerSubscription/SubscribeAlerts`	Anonymous	Registers email for automated job alerts; dispatches confirmation verification email.

7.5 Anti-Abuse, Rate-Limiting & Upload Security Controls

The public ingress perimeter enforces multi-layered defense-in-depth:

- **Sliding-Window IP Rate Limiter:** Limits requests to 10 submissions per minute per IP address. Rapid bursts trigger HTTP 429 Too Many Requests.
- **Google reCAPTCHA v3 Integration:** Candidate form submissions evaluate score thresholds; scores below 0.5 are immediately rejected.
- **Resume Magic Byte Validation:** Uploaded resume files must pass true binary header inspection (verifying `%PDF-1.x` signatures) rather than trusting MIME-type headers or file extensions. Maximum file size is strictly capped at 5 MB.

7.6 Chapter 7 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-07-001`	CRITICAL	Public resume upload allowed malicious script injection via crafted `.html` file masquerading with a `.pdf` extension.	Implement byte-array magic header inspection verifying `0x25, 0x50, 0x44, 0x46` bytes; sanitize uploaded file names to random GUIDs.
`DEF-07-002`	HIGH	Missing company code in public career route caused database query to search across all tenant databases, violating tenant boundary.	Enforce mandatory non-empty `companyCode` route constraint; return HTTP 400 Bad Request immediately if code is absent or invalid.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-07-01`	Public career page live.	Submit 15 rapid POST requests from identical IP within 30 seconds.	Requests 11 through 15 return HTTP 429 Rate Limit Exceeded.
`QA-07-02`	Candidate receives 30-minute upload token.	Attempt resume upload at `T + 31` minutes.	HTTP 401 Unauthorized; token expired; applicant prompted to re-authenticate.

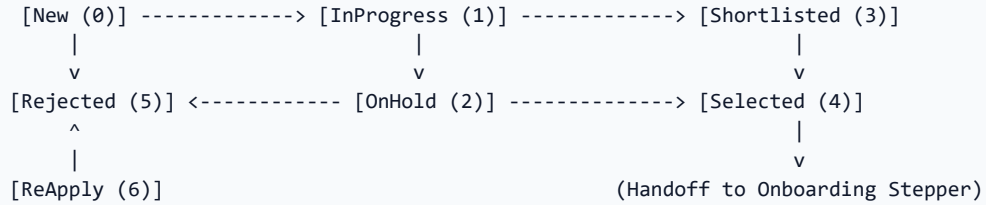
8. Application Processing, Candidate Pipeline & Resume Vault

8.1 7-Stage Candidate Pipeline State Machine

Every inbound application is captured as an `Applicant` entity associated with an immutable audit journal (`ApplicantLogs`). Recruiter evaluations, interview milestones, and hiring status transitions follow an explicit state machine governed by the `ActivityType` enum:

Pipeline Stage	Code	Operational Meaning & Transition Preconditions	Automated Side Effects
New	`0`	Initial application received from career portal or manual recruiter entry. Awaiting initial screening.	Application ack email sent; HR notification dispatched.
InProgress	`1`	Candidate assigned to recruiter; active evaluation, screening call, or technical test underway.	Logged in `ApplicantLogs`; visible in recruiter queue.
OnHold	`2`	Candidate qualified but requisition temporarily paused or alternative candidates prioritized.	Status email sent if configured; recruiter reminder set.
Shortlisted	`3`	Candidate passed preliminary screening; scheduled for formal technical/managerial interview loops.	Interview invitation email; calendar event generated.
Selected	`4`	Candidate cleared interview loop; formal offer extended, accepted, and counter-signed.	**Recruitment Handoff Trigger:** Prepares employee onboarding record.
Rejected	`5`	Candidate disqualified based on technical fit, experience, or compensation expectation mismatch.	Regret notification email sent; feedback archived.
ReApply	`6`	Candidate invited to re-enter evaluation cycle for alternative requisition or future talent pool.	Reset pipeline stage for new requisition link.

FIGURE 8.1: 7-STAGE CANDIDATE PIPELINE TRANSITION DIAGRAM



8.2 In-Browser PDF Resume Previewer & Text Tokenization Pipeline

To accelerate candidate triage, the recruiter interface embeds a secure PDF viewer using PDF.js. In the backend, incoming resumes are streamed through an Apache Tika / PDFSharp text extraction pipeline that extracts plain-text tokens, enabling full-text keyword searches across thousands of candidate CVs without downloading local files.

8.3 Dynamic Recruiter Email Templates & Variable Substitution

The recruitment communications engine dispatches automated notifications at each pipeline milestone. System templates support rich variable interpolation:

- `{CandidateName}`: Full legal or preferred name of the applicant.

- `{JobTitle}`: Designation specified on the originating requisition.
- `{CompanyName}`: Corporate tenant branding label.
- `{InterviewDate} / {InterviewTime}`: Formatted UTC timestamp normalized to candidate's localized timezone.
- `{MeetingLink}`: Secure video conference URL (Google Meet / Microsoft Teams).

8.4 Resume Vault & Secure Streaming Protocol

Applicant resumes contain highly sensitive Personally Identifiable Information (PII). The platform forbids direct file serving or public S3 bucket exposure. Resumes are stored in a protected storage vault and accessed exclusively through streaming endpoints that enforce tenant authorization and short-lived pre-signed URLs.

8.5 Recruitment to Onboarding Handoff Engine

When an applicant reaches the `Selected` stage, the system executes `RecruitmentHandoff`. Candidate profile attributes, contact numbers, email addresses, resume links, and agreed compensation packages are automatically packaged into an onboarding transfer payload. This initiates the employee onboarding workflow, preventing repetitive manual data entry and ensuring data fidelity.

8.6 Chapter 8 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-08-001`	HIGH	Transitioning applicant directly from `New` to `Selected` bypassed interview scorecard validation, allowing unauthorized hiring records.	Implement state machine transition guard in `ApplicantService`: moving to `Selected` requires at least one completed interview evaluation.
`DEF-08-002`	MEDIUM	Concurrent stage changes by multiple recruiters caused audit log entries in `ApplicantLogs` to be written out of chronological sequence.	Use MongoDB `$push` operator with UTC server timestamp and optimistic concurrency version check on the parent `Applicant` document.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-08-01`	Candidate in `Shortlisted` stage.	Recruiter moves candidate to `Selected`.	HTTP 200 OK; draft onboarding record initialized in `Onboarding` collection.
`QA-08-02`	Candidate already in `Rejected` state.	Recruiter attempts transition to `Shortlisted`.	HTTP 400 Bad Request; invalid state transition; must transition through `ReApply` first.

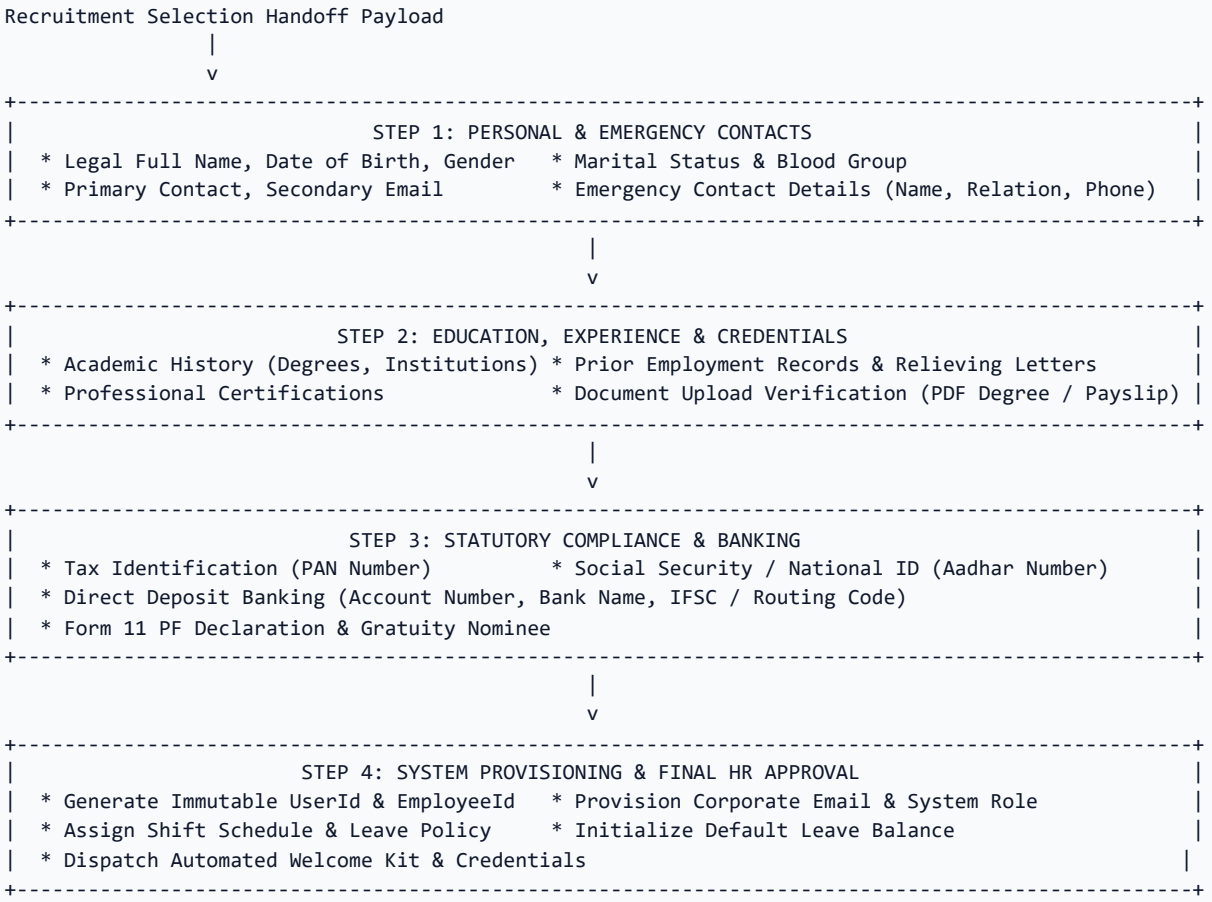
9. Employee Onboarding & 4-Step Account Provisioning Architecture

9.1 Business Drivers, Compliance Mandate & Architecture

Employee onboarding represents the critical bridge between candidate selection and active workforce productivity. Traditional manual onboarding processes suffer from missing statutory documents (tax IDs, bank credentials), delayed identity provisioning (email, intranet accounts), and uncoordinated IT asset deliveries. Incomplete statutory compliance on joining day exposes enterprises to severe labor audit penalties.

The Codeji Onboarding Module implements a 4-step progressive onboarding engine that guarantees complete, validated data collection before generating active employee accounts.

FIGURE 9.1: 4-STEP PROGRESSIVE ONBOARDING ARCHITECTURE & PROVISIONING FLOW



9.2 Statutory Verification Checklists & Document Compliance

During Step 3, HR operations officers execute an authoritative statutory checklist before clearing the case:

Compliance Document	Statutory Purpose	Verification Check Rule	Failure Remediation
PAN Card Copy	TDS Withholding (Income Tax Act)	Validate name matches candidate profile.	Reject Step 3; prompt re-upload.
Aadhar Card / National ID	Identity Proof & EPFO KYC Link	12-digit format; mask first 8 digits in UI.	Prompt clear copy showing full DOB.
Cancelled Cheque / Passbook	Salary Direct Deposit Account	Validate IFSC matches bank branch code.	Prompt official bank statement.
EPFO Form 11 Declaration	Provident Fund Membership KYC	Check previous UAN declaration.	Link existing UAN or auto-generate.

9.3 IT Asset Pre-Provisioning & Welcome Kit Staging

Concurrently with Step 3 statutory verification, the onboarding engine triggers IT infrastructure provisioning hooks:

- **Corporate Email Provisioning:** Generates standardized mailbox (e.g. `firstname.lastname@company.com`) via Google Workspace or Microsoft Graph API.
- **Role & Directory Assignment:** Pre-allocates default security group memberships and VPN certificates based on the employee's assigned designation and department.
- **Hardware Staging:** Creates an asset reservation in the IT inventory ledger (Laptop serial number, monitor tag) scheduled for courier dispatch or desk delivery on joining day.

9.4 Onboarding Lifecycle State Machine

The onboarding progression is modeled as an aggregate entity (`EmployeeOnboarding`) governed by strict state transitions:

- **Draft:** Step-by-step data collection active. Steps 1 through 3 can be edited progressively by candidate or HR operator.
- **InVerification:** Candidate submitted all 3 steps; HR and compliance officers reviewing uploaded documents and bank details.
- **PendingApproval:** Statutory verification complete; awaiting final HR Director or Department Head confirmation.
- **Completed:** Final approval executed. System generates active `EmpUser` record, assigns default leave policy, creates initial shift schedule, and marks candidate as an active employee.
- **Abandoned:** Candidate failed to report or declined offer prior to joining date. Process terminated.

9.5 Onboarding API Contracts

HTTP Method	Endpoint Route	Auth / Permission	Contract Details & Constraints
`GET`	`/api/admin/onboarding/cases`	`Onboarding.View`	Returns all active onboarding cases with current step progress and verification status.
`POST`	`/api/admin/onboarding/step1`	`Onboarding.Edit`	Saves personal and emergency contact details; executes regex phone/email validation.
`POST`	`/api/admin/onboarding/step2`	`Onboarding.Edit`	Persists educational history and employment certificates; streams document attachments.
`POST`	`/api/admin/onboarding/step3`	`Onboarding.Edit`	Saves statutory PAN, Aadhar, and banking details; validates IFSC checksum.
`POST`	`/api/admin/onboarding/complete`	`Onboarding.Approve`	Executes account provisioning; generates `EmpUser`, initializes leave quota, sends welcome email.

9.6 Chapter 9 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-09-001`	CRITICAL	Onboarding completion endpoint generated new `EmpUser` record without wrapping in a distributed transaction, resulting in orphaned user accounts if leave policy assignment threw an exception.	Implement multi-document MongoDB transaction wrapping user creation, shift assignment, and leave quota initialization; execute rollback on any step failure.
`DEF-09-002`	HIGH	Lack of bank account format verification permitted corrupted bank account strings, which later failed during automated monthly payroll direct deposit export.	Implement server-side regex and modulus-11 checksum validation on bank account numbers and IFSC codes in Step 3 handler.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-09-01`	Onboarding at Step 2; Step 3 statutory data not submitted.	Attempt to invoke `POST /api/admin/onboarding/complete`.	HTTP 400 Bad Request; onboarding blocked until all statutory compliance steps verified.
`QA-09-02`	All 4 onboarding steps verified and approved.	Execute `POST /api/admin/onboarding/complete`.	HTTP 200 OK; new employee appears in directory, default leave balances populated, status is Active.

PART III: PEOPLE OPERATIONS & EMPLOYEE MASTER

Employee Master, Unified Profile, Education & Lifecycle Progression

Authoritative specification of the employee core aggregate (EmpUser), statutory compliance frameworks (PF, ESI, PAN, Gratuity), educational credential verification pipelines, probation evaluation milestones, 9-box appraisal rubrics, internal transfer mechanics, and retroactive arrears computation.

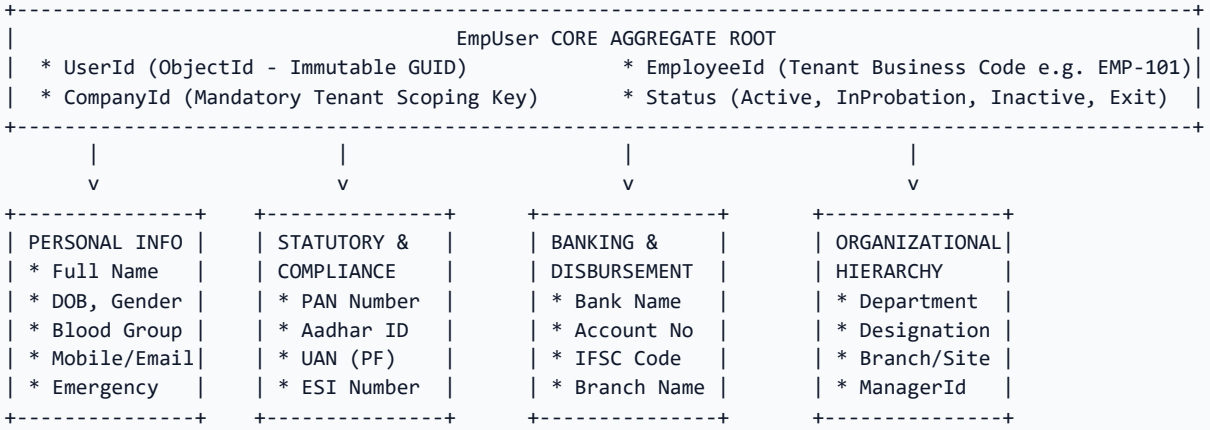
10. Employee Master, Unified Profile & Statutory Compliance Architecture

10.1 Business Drivers, Central Master Repository & Core Value

In any enterprise organization, the Employee Master repository is the single source of truth upon which all transactional modules depend. Discrepancies in personal details, unverified tax identifiers, or out-of-date banking information ripple outward: causing failed salary disbursements, inaccurate tax filings, erroneous biometric punch matching, and severe non-compliance penalties from government regulatory bodies.

The Codeji Employee Master is architected as a cohesive aggregate (`EmpUser`) that encapsulates core identity, statutory credentials, organizational relationships, and profile history while maintaining strict tenant isolation and auditability.

FIGURE 10.1: UNIFIED EMPLOYEE PROFILE ARCHITECTURAL MODEL



10.2 Domain Model & C# Entity Definitions

// Core Employee Aggregate Root Definition in CMS-Backend-Core [BsonCollection("EmpUser")] public class EmpUser : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string UserId { get; set; } = string.Empty; [BsonElement("CompanyId")] public string CompanyId { get; set; } = string.Empty; [BsonElement("EmployeeId")] public string EmployeeId { get; set; } = string.Empty; [BsonElement("JoiningDate")] public DateTime JoiningDate { get; set; } [BsonElement("ConfirmationDate")] public DateTime? ConfirmationDate { get; set; } [BsonElement("Status")] public EmploymentStatus Status { get; set; } = EmploymentStatus.InProbation; [BsonElement("ManagerId")] public string? ManagerId { get; set; } [BsonElement("DepartmentId")] public string

DepartmentId { get; set; } = string.Empty; [BsonElement("DesignationId")] public string DesignationId { get; set; } = string.Empty; [BsonElement("BranchId")] public string BranchId { get; set; } = string.Empty; [BsonElement("PersonalDetails")] public PersonalDetailsModel PersonalDetails { get; set; } = new(); [BsonElement("StatutoryDetails")] public StatutoryDetailsModel StatutoryDetails { get; set; } = new(); [BsonElement("BankDetails")] public BankDetailsModel BankDetails { get; set; } = new(); [BsonElement("EmergencyContacts")] public List<EmergencyContactModel> EmergencyContacts { get; set; } = new(); [BsonElement("NoticePeriodDays")] public int NoticePeriodDays { get; set; } = 30; [BsonElement("Version")] public int Version { get; set; } = 1; }

10.3 MongoDB Sub-Document Schemas & Data Constraints

Sub-Document / Field	BSON / C# Type	Validation Rule / Invariant	Operational Meaning
`PersonalDetails.FullName`	`string`	Mandatory; 2 to 100 characters	Official employee name per government identity documents.
`PersonalDetails.DOB`	`DateTime`	Age must be >= 18 years at JoiningDate	Birth date for statutory age verification and superannuation.
`StatutoryDetails.PAN`	`string`	Regex: `^[A-Z]{5}[0-9]{4}[A-Z]{1}$`	Permanent Account Number for TDS withholding & 26AS reporting.
`StatutoryDetails.UAN`	`string`	Exact 12 digits numeric	Universal Account Number for EPFO provident fund tracking.
`StatutoryDetails.ESICNumber`	`string`	Exact 17 digits numeric	Employee State Insurance identity number.
`BankDetails.AccountNumber`	`string`	9 to 18 digits numeric; encrypted at rest	Direct deposit disbursement account identifier.
`BankDetails.IFSCCode`	`string`	Regex: `^[A-Z]{4}0[A-Z0-9]{6}$`	RBI bank branch clearing routing code.

10.4 Statutory Compliance Deep Dive: Indian Regulatory Frameworks

The Codeji HRIS implements strict adherence to statutory labor and taxation acts, automating monthly calculation and reporting across four primary domains:

STATUTORY RULES: PROVIDENT FUND (EPFO) CONTRIBUTION FORMULATION

```
Statutory Wage Ceiling = ₹15,000 per month (PF Basic)
Eligible PF Wage = MIN(Employee.BasicSalary, StatutoryWageCeiling) [or Actual Basic if Uncapped Opt-In]

Employee Contribution = 12.0% of Eligible PF Wage
Employer Contribution Breakdown:
* Employees' Pension Scheme (EPS) = 8.33% of Eligible PF Wage (Subject to statutory cap ₹1,250/month)
* Employees' Provident Fund (EPF) = 12.0% - EPS Contribution (Balancing figure: ~3.67%)
* EDLI Contribution (A/C 21) = 0.50% of Eligible PF Wage (Subject to cap ₹75/month)
* EPF Admin Charges (A/C 02) = 0.50% of Total PF Wages (Minimum ₹500/month per establishment)
```

Statutory Component	Threshold	Employee Rate	Employer Rate	Compliance Reference
Employees' Provident Fund (EPF)	₹15,000 / month Basic	12.0% of Basic	3.67% EPF + 8.33% EPS	Employees' Provident Funds & Misc Provisions Act 1952
Employees' State Insurance (ESIC)	₹21,000 / month Gross	0.75% of Gross Wages	3.25% of Gross Wages	Employees' State Insurance Act 1948
Tax Deducted at Source (TDS)	Annual Exemption Limit	Per Slab / Regime (New vs Old)	N/A (Withholding Agent)	Section 192, Income Tax Act 1961
Payment of Gratuity	5 Years Continuous Service	N/A (Fully Employer Funded)	4.81% Accrual Provision	Payment of Gratuity Act 1972 (Max ₹20,00,000)

10.5 Enterprise Document Vault & KYC Cryptographic Security

To comply with Digital Personal Data Protection (DPDP) standards and UIDAI guidelines, the Employee Master implements defense-in-depth data protection:

- **Aadhaar Masking Pipeline:** UIDAI strictly prohibits storing unmasked 12-digit Aadhaar numbers in databases. When an employee uploads an Aadhaar card, a background OCR processor inspects the document, masks the first 8 digits, and writes only the masked string `XXXXXXXX1234` to MongoDB.
- **PAN Format & NSDL Verification:** The 10-character alphanumeric PAN is validated against the structure: 5 letters (4th letter entity type 'P' for Individual, 5th letter employee surname initial), 4 numeric digits, 1 check digit. The system integrates with NSDL/Protean API to confirm PAN-Aadhaar linking status. Under Section 206AA, invalid or unlinked PANs trigger automatic punitive TDS withholding at a flat 20%.
- **Automated Penny-Drop Bank Verification:** To eliminate salary disbursement bounces caused by mistyped account numbers or fraudulent IFSC codes, the system executes an automated NPCI penny-drop transfer (₹1.00 credit) via banking webhooks. The webhook validates that the registered bank account holder name matches `PersonalDetails.FullName` with a Jaro-Winkler string similarity threshold of >= 0.85.

10.6 Statutory Regulatory Compliance Calendar

Statutory Obligation	Governing Authority	Statutory Filing Due Date	System Automation Output
EPFO Return (ECR)	Employees' Provident Fund Org	15th of every calendar month	Generates monthly ECR text file with UAN matching.
ESIC Contribution	Employees' State Insurance Corp	15th of every calendar month	Exports monthly Challan wage upload sheet.
TDS Deposit (Challan 281)	Income Tax Department	7th of every calendar month	Itemizes monthly tax withheld by PAN.
Quarterly TDS (Form 24Q)	National Securities Depository (NSDL)	31st following quarter end	Prepares FVU-compliant text file for return submission.

10.7 Employee Master API Contracts

HTTP Method	Endpoint Route	Auth / Permission	Contract Details & Constraints
`GET`	`/api/admin/employees`	`Employee.View`	Returns paginated list with department, designation, status, and shift filters.
`GET`	`/api/admin/employees/{id}`	`Employee.View`	Returns complete 360-degree employee profile document including statutory info.
`PUT`	`/api/admin/employees/{id}/statutory`	`Employee.Edit`	Updates PAN, UAN, ESI details; requires audit reason and version check.
`PUT`	`/api/admin/employees/{id}/bank`	`Employee.Edit`	Updates bank account details; dispatches audit alert to payroll administrators.
`GET`	`/api/profile/me`	Authenticated User	Self-service profile read for logged-in employee derived from token claims.

10.8 Chapter 10 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-10-001`	CRITICAL	Concurrent updates to employee contact details and reporting manager overwritten each other due to full-document replacement rather than granular field-level updates.	Refactor repository update methods to use MongoDB `Builders<EmpUser>.Update.Set()` with optimistic `Version` increment.
`DEF-10-002`	HIGH	Unmasked PAN and Aadhar numbers returned in general employee directory search responses, exposing sensitive PII to unauthorized internal viewers.	Introduce projection DTOs (`EmployeeListDto`) that omit statutory and banking sub-documents; statutory fields restricted strictly to authorized payroll endpoints.

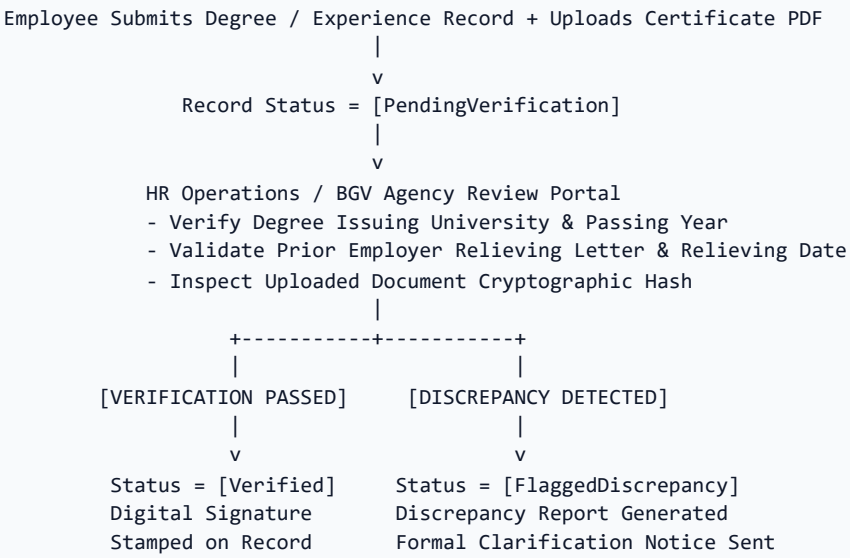
Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-10-01`	Standard employee logged in.	Invoke `GET /api/admin/employees`.	HTTP 403 Forbidden; directory administrative views require `Employee.View` claim.
`QA-10-02`	HR Admin updates PAN number with invalid format (e.g. "ABCDE123").	Dispatch `PUT /api/admin/employees/{id}/statutory`.	HTTP 400 Bad Request; fails regex validation `[A-Z]{5}[0-9]{4}[A-Z]{1}`.

11. Educational Records, Professional Certifications & Work History

11.1 Purpose, Compliance Audits & Verification Workflow

Enterprises operating in regulated sectors (finance, healthcare, software consulting) face stringent regulatory audits requiring verified proof of employee educational credentials and prior work experience. The Education & Work History module provides a structured, verifiable digital repository for academic degrees, professional certifications (e.g. AWS, PMP, CISSP), and prior employer relieving documents.

FIGURE 11.1: CREDENTIAL VERIFICATION & BGV AUDIT PIPELINE



11.2 Education & Experience Domain Entities

// Educational Credential Sub-Document Model public class EducationRecordModel { [BsonId] public string RecordId { get; set; } = Guid.NewGuid().ToString(); public string DegreeName { get; set; } = string.Empty; // e.g. "B.Tech Computer Science" public string InstitutionName { get; set; } = string.Empty; public string UniversityBoard { get; set; } = string.Empty; public int YearOfPassing { get; set; } public decimal PercentageCgpa { get; set; } public string DocumentStoragePath { get; set; } = string.Empty; public string DocumentHashSha256 { get; set; } = string.Empty; public VerificationStatus Status { get; set; } = VerificationStatus.PendingVerification; public string? VerifiedByUserId { get; set; } public DateTime? VerifiedAt { get; set; } public string? DiscrepancyNotes { get; set; } } // Work History Sub-Document Model public class WorkExperienceModel { [BsonId] public string RecordId { get; set; } = Guid.NewGuid().ToString(); public string OrganizationName { get; set; } = string.Empty; public string Designation { get; set; } = string.Empty; public DateTime StartDate { get; set; } public DateTime EndDate { get; set; } public decimal LastDrawnAnnualCtc { get; set; } public string RelievingLetterStoragePath { get; set; } = string.Empty; public string ExperienceCertificateStoragePath { get; set; } = string.Empty; public VerificationStatus Status { get; set; } = VerificationStatus.PendingVerification; public string? VerifiedByUserId { get; set; } public DateTime? VerifiedAt { get; set; } }

11.3 Multi-Tier Document Upload & Verification Status Matrix

Verification Stage	Triggering Event	System Validation Rule	Operational Action Required
`PendingVerification`	Employee completes document upload.	PDF format only; binary header check; size <= 5MB.	Queued in HR / BGV operational verification workbench.
`InProgress`	Reviewer opens document or API dispatches to BGV agency.	Lock record against employee modification during review.	BGV agency performs university / previous employer inquiries.
`Verified`	Reviewer approves credential authenticity.	Record marked immutable; cryptographic signature stamped.	Employee profile displays green "Verified Credential" badge.
`FlaggedDiscrepancy`	Reviewer detects falsified degree, GPA mismatch, or tenure overlap.	Audit log captures discrepancy classification and reviewer comments.	Automated escalation to HR Business Partner; formal show-cause alert.

11.4 Background Verification (BGV) Pipeline & Discrepancy Escalation

To eliminate manual verification bottlenecks, the platform implements a 3-tier background verification pipeline:

- **Tier 1: Automated Document Inspection:** On document upload, the server executes OCR text extraction and SHA-256 binary hash validation, preventing re-upload of altered or previously rejected credentials.
- **Tier 2: External BGV Agency API Integration:** The system dispatches webhook notifications containing candidate credentials to third-party verification partners (e.g. AuthBridge, IDfy). Webhook responses automatically transition status to `Verified` or `FlaggedDiscrepancy`.
- **Tier 3: Institutional Discrepancy Dispute Protocol:** If a discrepancy is logged, the employee has a 7-day statutory window to submit original transcripts or registrar certification. Failure to resolve transitions the profile to `IntegrityReview`, locking self-service capabilities.

11.5 Education & Experience API Contracts

HTTP Method	Endpoint Route	Auth / Permission	Contract Details & Constraints
`POST`	`/api/profile/me/education`	Authenticated User	Submits degree record with multipart PDF certificate; size limit 5MB.
`POST`	`/api/profile/me/experience`	Authenticated User	Submits prior employment record with relieving and experience letters.
`PUT`	`/api/admin/employees/{id}/education/{recId}/verify`	`Employee.VerifyCredentials`	Approves or flags educational credential; stamps verification timestamp.
`PUT`	`/api/admin/employees/{id}/experience/{recId}/verify`	`Employee.VerifyCredentials`	Approves or flags prior work experience; cross-references UAN service history.

11.6 Chapter 11 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-11-001`	HIGH	Employees were able to delete education records after they were marked as `Verified` by HR, allowing deletion of audited compliance data.	Enforce immutability guard: records with `VerificationStatus == Verified` cannot be deleted or modified by self-service users; changes require HR administrator override.
`DEF-11-002`	MEDIUM	Expiring professional certifications did not trigger renewal notification alerts, leading to compliance lapses during client security audits.	Implement background daily scheduled worker `CertificationExpiryWorker` that dispatches email and SignalR alerts 60, 30, and 7 days prior to expiry.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-11-01`	Education record in `Verified` status.	Employee attempts deletion via `DELETE /api/profile/me/education/{id}`.	HTTP 400 Bad Request; locked verified compliance record cannot be deleted.
`QA-11-02`	Experience record submitted with `EndDate < StartDate`.	Submit record payload to API.	HTTP 400 Bad Request; validation error: End date cannot precede Start date.

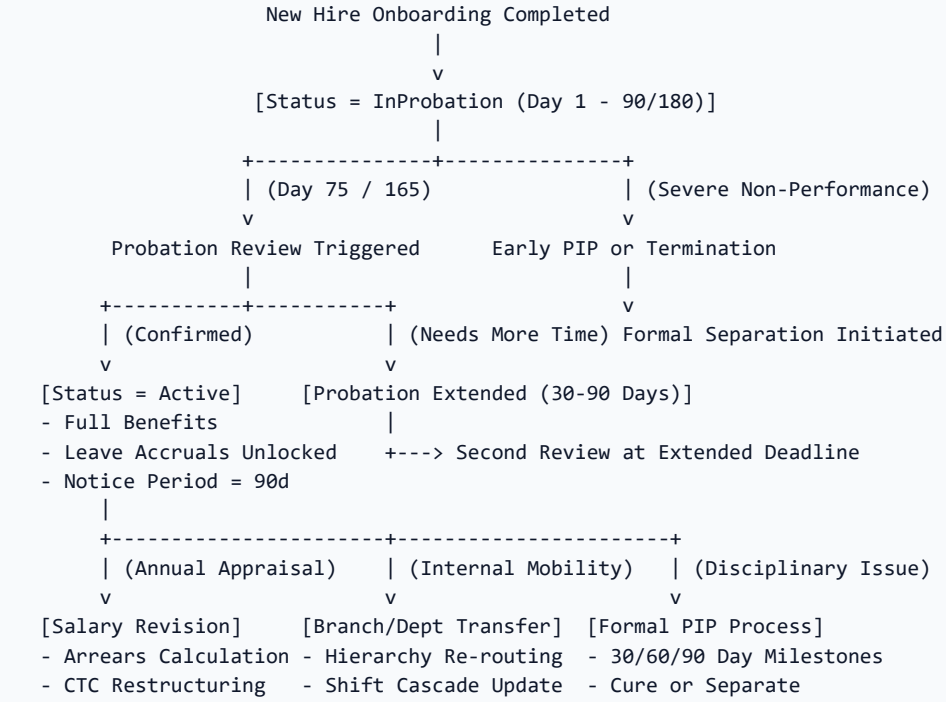
12. Employee Lifecycle Management (Probation, Confirmation & Transfers)

12.1 Lifecycle State Transitions & Milestone Automation

An employee's relationship with the enterprise evolves through distinct lifecycle milestones: probation evaluation, confirmation, periodic appraisals, internal department or branch transfers, and eventual offboarding. Tracking these milestones manually leads to missed probation end dates, unauthorized salary increments, and unsynchronized shift assignments.

The Lifecycle Management engine formalizes these transitions into an automated workflow engine:

FIGURE 12.1: EMPLOYEE LIFECYCLE MILESTONE STATE MACHINE



12.2 Probation Evaluation Workflow & 9-Box Appraisal Grid Matrix

To avoid default confirmations without performance evaluation, the platform requires managers to complete a structured 9-box performance vs. potential appraisal before confirmation approval:

9-Box Grid Quadrant	Performance / Potential Rating	System Milestone Action	Probation Confirmation Outcome
Star (1A)	High Performance / High Potential	Fast-track promotion consideration; equity grant.	Immediate Confirmation with distinction.
High Performer (1B)	High Performance / Medium Potential	Key contributor retention bonus; role expansion.	Standard Confirmation approved.
High Potential (2A)	Medium Performance / High Potential	Targeted technical mentoring and leadership track.	Standard Confirmation approved.
Core Player (2B)	Medium Performance / Medium Potential	Standard development plan; ongoing feedback.	Standard Confirmation approved.
Inconsistent Player (2C)	Medium Performance / Low Potential	Coaching on specific performance deliverables.	Probation Extended (30-60d) with target KPIs.
Risk / Underperformer (3C)	Low Performance / Low Potential	Performance Improvement Plan (PIP) or exit.	Probation Extended or formal separation initiated.

12.3 Departmental & Branch Transfer Mechanics

When an employee transfers between departments or branches, the system guarantees atomic transition of operational dependencies:

- **Reporting Hierarchy:** `ManagerId` updated to new department head or team lead. Outstanding leave/WFH approval requests automatically re-routed to new manager.
- **Shift Re-evaluation:** If employee relied on Department Default Shift, the system re-evaluates the shift precedence cascade, assigning the new department's shift without breaking historical attendance records.
- **Audit Journaling:** Every transfer creates an immutable record in `LifecycleAuditHistory` capturing OldDepartment, NewDepartment, OldDesignation, NewDesignation, EffectiveDate, ApprovedBy, and TransferReason.

12.4 Performance Improvement Plan (PIP) 30/60/90 Day Governance

Employees flagged as underperforming are transitioned to formal PIP governance:

- **Milestone Check-Ins:** Mandatory bi-weekly review meetings logged between manager and employee.
- **Objective Targets:** Measurable key performance indicators (KPIs) recorded in the PIP aggregate.
- **PIP Outcome:** Successful completion restores active confirmed status; unsuccessful completion triggers formal HR-initiated separation workflow.

12.5 Compensation Re-Basing, Promotion & Retroactive Salary Arrears Engine

When an employee receives a merit promotion or annual increment, the effective date often precedes the actual administrative approval date (e.g. appraisal approved in June, effective retrospectively from April 1st). The Codeji HRIS features a deterministic arrears calculation engine:

MATHEMATICAL FORMULATION: RETROACTIVE SALARY ARREARS CALCULATION

```
Let EffectiveDate = 2026-04-01, ExecutionMonth = 2026-06 (June 2026)
Retrospective Arrears Months = [April 2026, May 2026]

For each retroactive month M in Arrears Months:
* Let OriginalNetPaid = MonthlyPayroll(M).NetSalaryPayable
* Re-evaluate GrossSalary using NewSalaryStructure:
```

```
- NewBasic = NewSalaryStructure.BasicPayable(M.ActiveDays, M.LossOfPayDays)
- NewHRA = NewSalaryStructure.HRAPayable(M.ActiveDays, M.LossOfPayDays)
- NewSpecial = NewSalaryStructure.SpecialAllowance(M.ActiveDays, M.LossOfPayDays)
* Recompute Statutory Deductions:
- NewPF = CalculateEPF(NewBasic)
- NewPT = CalculatePT(NewGross)
- ArrearNetDifference(M) = (NewGross - (NewPF + NewPT)) - OriginalNetPaid

TotalSalaryArrears = SUM(ArrearNetDifference(M)) across all retroactive months.
TotalSalaryArrears is credited as a discrete taxable earning line item in June 2026 payroll.
```

// Domain Model for Salary Revision & Historical Compensation Tracking [BsonCollection("SalaryRevisionHistory")] public class SalaryRevisionHistory : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string RevisionId { get; set; } = string.Empty; public string CompanyId { get; set; } = string.Empty; public string UserId { get; set; } = string.Empty; public DateTime EffectiveDate { get; set; } public DateTime ApprovedDate { get; set; } public string ApprovedByUserId { get; set; } = string.Empty; public decimal PreviousAnnualCtc { get; set; } public decimal RevisedAnnualCtc { get; set; } public decimal IncrementPercentage { get; set; } public string RevisionReason { get; set; } = "Annual Appraisal"; // Promotion, Market Correction, Appraisal public string? PromotionDesignationId { get; set; } public decimal ComputedRetroactiveArrears { get; set; } public bool IsArrearDisbursed { get; set; } = false; public string? DisbursedInPayrollMonth { get; set; } }

12.6 Chapter 12 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-12-001`	HIGH	Department transfer immediately severed manager visibility on historical attendance and leave records, causing manager approval queues to orphan pending items.	Implement retroactive approval re-assignment: on transfer execution, query all pending requests for the employee and re-assign `ReviewerId` to the new reporting manager.
`DEF-12-002`	MEDIUM	Probation confirmation date did not automatically adjust the employee's notice period configuration from 30 days to 90 days.	Attach domain event listener `EmployeeConfirmedDomainEvent` that updates `NoticePeriodDays` in the employee profile upon confirmation approval.
`DEF-12-003`	HIGH	Retroactive salary arrears disbursed in current month did not recompute employer PF contribution differentials, leading to ECR reconciliation mismatch.	Implement explicit arrears reconciliation line item in monthly ECR export routine, allocating arrear Basic wages to past contribution months.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-12-01`	Employee in `InProbation` status.	HR approves performance confirmation appraisal.	Status transitions to `Confirmed`; confirmation letter generated; leave policy updated.
`QA-12-02`	Employee has pending leave request with Manager A.	HR transfers employee to Department B under Manager B.	Pending leave request reviewer updated to Manager B; Manager B receives SignalR alert.
`QA-12-03`	Salary increment effective 2 months ago approved today.	Execute current month payroll preview.	Salary arrears calculated exactly matching 2-month gross differential less deductions.

PART IV: TIME, ATTENDANCE & WORK ARRANGEMENTS

Work From Home (WFH), Attendance Tracking, Source Reconciliation & Monthly Locking

In-depth technical architecture of daily punch analytics, biometric hardware push listeners, shift calculation engines, 4-tier shift precedence cascades, source-linked attendance provenance (SourceType/SourceId), LHD/ED penalty deductions, exception auditing, and the payroll locking boundary.

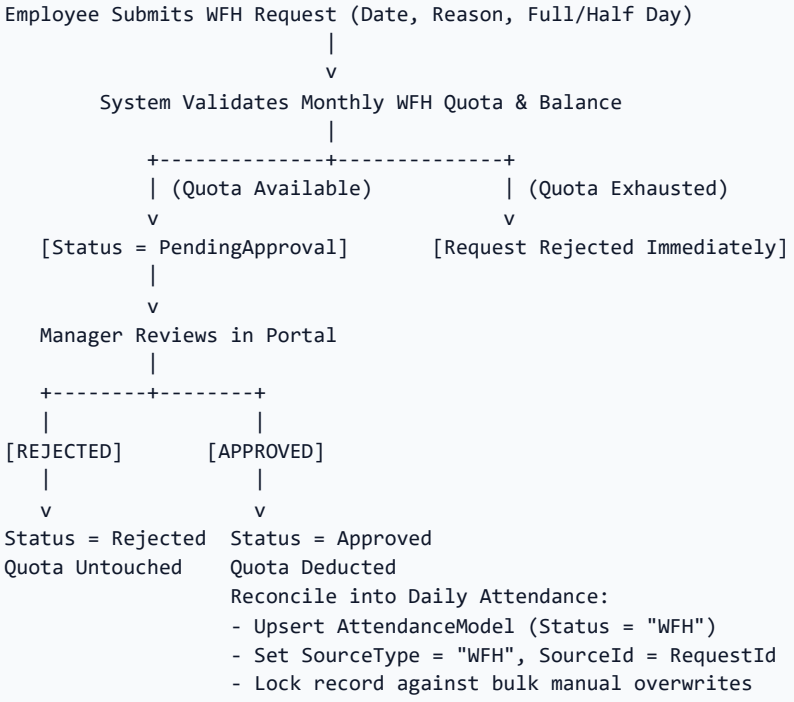
13. Work From Home (WFH) Management Engine & Quota Architecture

13.1 Business Drivers, Hybrid Workforce Governance & Policy Rules

In the post-pandemic enterprise landscape, hybrid work policies are universal. However, unstructured remote work without tracking leads to severe operational visibility loss, attendance discrepancies, and abuse of remote allowances. The WFH Management Engine establishes a rigorous, policy-backed remote work lifecycle that balances employee flexibility with operational accountability.

Every company maintains configurable WFH quotas (e.g., maximum 4 remote days per month per employee). WFH requests support full-day and half-day increments, require manager approval, and directly reconcile into daily attendance as verified remote working records.

FIGURE 13.1: WFH REQUEST & ATTENDANCE RECONCILIATION LIFECYCLE



13.2 Remote Work IP Whitelisting & Geolocation Logging

When an employee checks in remotely under an approved WFH allocation, the system captures network security telemetry to verify location compliance:

- **Client Public IP Logging:** Captured in `AttendanceModel.CheckInIp`; validated against known company VPN egress ranges or allowed corporate subnets if configured.
- **User-Agent & Browser Fingerprint:** Stored in audit logs to detect automated script clock-ins or unauthorized third-party device clocking.
- **Remote Punch Confirmation:** Pushes live notification to the employee's reporting manager via SignalR, confirming remote shift commencement.

13.3 Manager Approval SLA & Escalation Protocol

To prevent remote requests from remaining pending indefinitely, the system enforces automated SLA timers:

- **24-Hour Reminder:** Background worker sends reminder notification to manager if request remains unreviewed 24 hours after submission.
- **48-Hour Auto-Escalation:** If still pending at 48 hours, request auto-escalates to Department Head or HR Business Partner with audit notation.

13.4 MongoDB Schema & Entity Definitions

Field Name	BSON / C# Type	Validation Rule / Invariant	Operational Meaning
`WfhRequestId`	`string / ObjectId`	Mandatory Primary Key	Unique identifier for the remote work request.
`CompanyId`	`string`	Mandatory Tenant Scope	Tenant boundary identifier.
`UserId`	`string`	Must match valid employee	Employee submitting the request.
`RequestDate`	`DateTime`	Cannot be in closed payroll month	Calendar date for which WFH is requested.
`DayPortion`	`DayPortion (enum)`	`FullDay (1.0)` or `HalfDay (0.5)`	Deduction quantum against monthly allowance.
`Status`	`WfhStatus (enum)`	`Pending`, `Approved`, `Rejected`, `Cancelled`	Current lifecycle state of the request.

13.5 WFH Management API Contracts

HTTP Method	Endpoint Route	Auth / Permission	Contract Details & Constraints
`POST`	`/api/wfh/request`	Authenticated User	Submits remote work request; checks quota balance; returns 400 if exceeded.
`GET`	`/api/wfh/me`	Authenticated User	Returns employee's WFH requests and remaining monthly quota.
`PUT`	`/api/wfh/{id}/review`	`WFH.Review`	Manager approves or rejects request; triggers attendance reconciliation.

13.6 Chapter 13 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-13-001`	HIGH	Two concurrent WFH requests submitted simultaneously bypassed monthly quota check, allowing 5 WFH days when policy maximum was 4.	Implement distributed locking on employee quota or transactional MongoDB write with pre-condition check on consumed days.
`DEF-13-002`	MEDIUM	Approved WFH request cancellation failed to clear the attendance record, leaving employee marked as WFH instead of Absent.	Attach reconciliation compensation action to WFH cancellation handler: verify `SourceId` matches before resetting attendance status.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-13-01`	Employee has consumed 4 of 4 WFH days.	Submit request for 5th WFH day.	HTTP 400 Bad Request; "Monthly WFH quota of 4 days exhausted".
`QA-13-02`	Manager approves WFH for 2026-09-25.	Query `GET /api/admin/attendance` for that date.	Attendance row exists with `Status = "WFH"` and `SourceType = "WFH"`.

14. Daily Attendance Tracking, Punch Analytics & Shift Calculation Engine

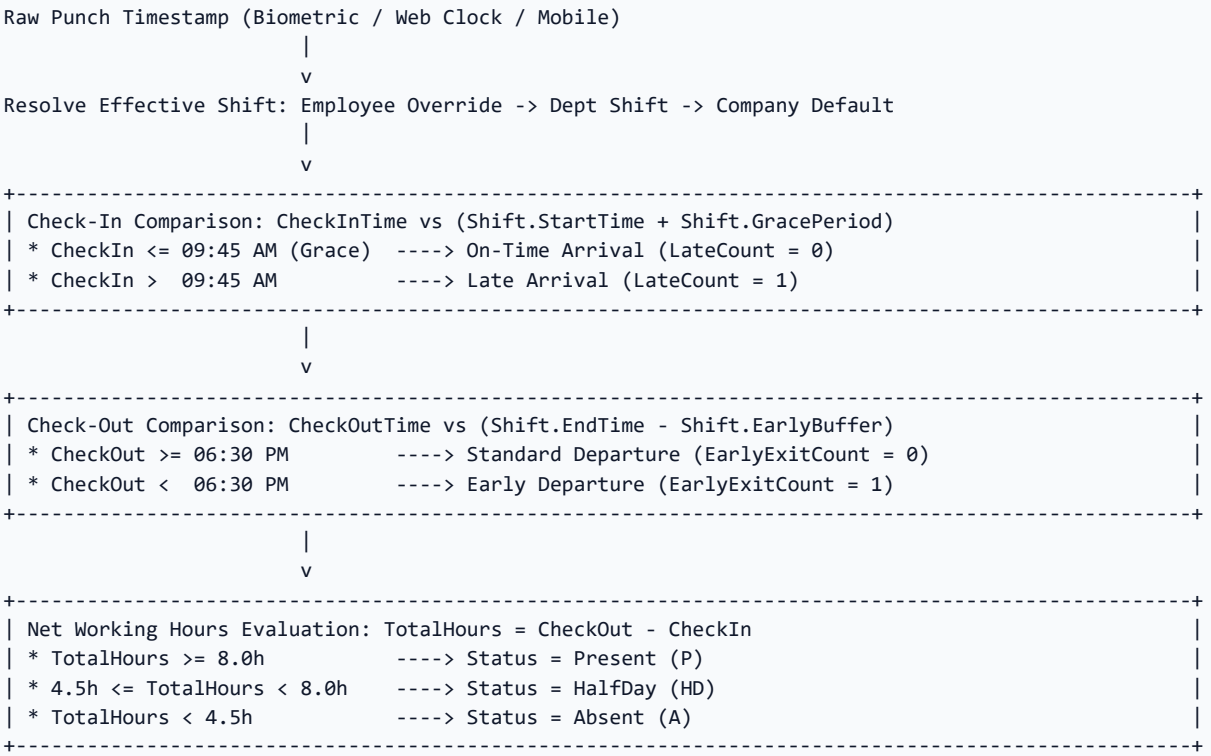
14.1 Daily Attendance Engine & Punch Processing Mechanics

The Attendance module is the core operational engine determining paid and unpaid workforce days. Every daily record represents the synthesis of physical punch events, biometric logs, mobile clock-ins, and shift comparison algorithms.

The daily evaluation pipeline executes deterministic calculations against the employee's effective shift:

- **Arrival Evaluation:** If `CheckInTime > Shift.StartTime + Shift.GracePeriodMinutes`, increment `LateCount = 1`.
- **Departure Evaluation:** If `CheckOutTime < Shift.EndTime - Shift.EarlyExitBufferMinutes`, increment `EarlyExitCount = 1`.
- **Duration Evaluation:** `TotalHours = (CheckOutTime - CheckInTime).TotalHours`.
- **Status Derivation:**
  - If `TotalHours >= Shift.FullDayThresholdHours` (e.g. 8.0h) → Status = `Present (P)`.
  - If `Shift.HalfDayThresholdHours <= TotalHours < Shift.FullDayThresholdHours` → Status = `HalfDay (HD)`.
  - If `TotalHours < Shift.HalfDayThresholdHours` or No Punch → Status = `Absent (A)`.

FIGURE 14.1: DAILY ATTENDANCE PUNCH EVALUATION ENGINE



14.2 Biometric Hardware Push SDK & Edge Device Ingestion Architecture

To support physical biometric turnstiles and optical fingerprint/facial recognition terminals (ZKTeco, eSSL, Matrix, Hikvision), the system implements an enterprise ADMS (Automatic Data Master Server) ingestion gateway:

FIGURE 14.2: BIOMETRIC HARDWARE EDGE PUSH LISTENER & INGESTION QUEUE



The push ingestion pipeline processes terminal events in real time:

- **Terminal Handshake & Heartbeat:** Clocks dispatch `GET /iclock/cdata?SN={Serial}` every 60 seconds. The server responds with `OK` or pushes configuration commands (e.g. user bio-template sync).
- **Push Payload Parsing:** Turnstiles post raw tab-delimited strings: `USERID=1042 CHECKTIME=2026-09-18 09:14:22 CHECKTYPE=0`. The gateway parses these into strongly typed `RawPunchRecord` models.
- **60-Second Sliding Window Deduplication:** Biometric clocks register multiple scans when employees panic-scan. If an incoming punch for an employee matches a recorded punch within 60 seconds, it is discarded as a duplicate.
- **Offline Replay Ingestion:** In the event of an ISP failure, edge terminals buffer punches locally. Upon internet restoration, devices replay the entire offline backlog. The worker processes records asynchronously without triggering false late-mark alerts.

14.3 Core Attendance Domain Models & C# Entity Definitions

// Core Attendance Aggregate Document in CMS-Backend-Core [BsonCollection("Attendance")] [BsonIgnoreExtraElements] public class AttendanceModel : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string Id { get; set; } = string.Empty; public string CompanyId { get; set; } = string.Empty; public string UserId { get; set; } = string.Empty; public DateTime Date { get; set; } // Truncated to 00:00:00 UTC public string Status { get; set; } = "Present"; // Present, Absent, HalfDay, Leave, WFH, Holiday, WeeklyOff public DateTime? CheckInTime { get; set; } public DateTime? CheckOutTime { get; set; } public decimal? TotalHours { get; set; } public int LateCount { get; set; } = 0; public int EarlyExitCount { get; set; } = 0; public bool IsPenaltyWaived { get; set; } = false; public string? WaivedByUserId { get; set; } public string? WaiverReason { get; set; } // Provenance Tracking public string SourceType { get; set; } = "MANUAL"; // BIOMETRIC, WEB_CLOCK, MOBILE, LEAVE, WFH, MANUAL public string? SourceId { get; set; } // RequestId if LEAVE or WFH public int SourceVersion { get; set; } = 1; public bool IsRegularized { get; set; } = false; public string? RegularizationReason { get; set; } public string? CheckInIp { get; set; } public double? CheckInLatitude { get; set; } public double? CheckInLongitude { get; set; } } // Raw Biometric Ingestion Record [BsonCollection("RawPunchRecords")] public class RawPunchRecord { [BsonId] public string Id { get; set; } = Guid.NewGuid().ToString(); public string CompanyId { get; set; } = string.Empty; public string DeviceSerialNumber { get; set; } = string.Empty; public string BiometricEnrollmentId { get; set; } = string.Empty; public DateTime PunchTimestamp { get; set; } public int PunchDirection { get; set; } // 0: CheckIn, 1: CheckOut, 2: BreakOut, 3: BreakIn public DateTime IngestedAt { get; set; } = DateTime.UtcNow; public bool IsProcessed { get; set; } = false; }

14.4 Mobile Geofencing & Haversine Distance Formulation

For mobile field staff and branch employees clocking in via mobile devices, the system enforces geofencing validation using the spherical Haversine trigonometric formulation:

MATHEMATICAL FORMULATION: HAVERSINE GEOFENCING DISTANCE CALCULATION

```
Let (lat1, lon1) = Employee Device GPS Coordinates (in radians)
Let (lat2, lon2) = Branch Office Coordinates (in radians)
Let EarthRadius = 6,371,000 meters

dLat = lat2 - lat1
dLon = lon2 - lon1
a = sin²(dLat / 2) + cos(lat1) * cos(lat2) * sin²(dLon / 2)
c = 2 * atan2(√a, √(1 - a))
DistanceInMeters = EarthRadius * c

If DistanceInMeters <= Branch.GeofenceRadiusMeters (e.g. 150m) ----> Clock-In Permitted
Else ----> Reject Clock-In with HTTP 403: "Punch rejected: outside authorized work perimeter"
```

14.5 Shift Precedence Cascade & Overnight Shift Mechanics

Enterprise organizations frequently employ dynamic shift assignments across shifts, departments, and project teams. When the system evaluates daily punches, it traverses a 4-tier shift precedence hierarchy:

Precedence Tier	Shift Level	Evaluation Condition	Resolution Action
Level 1 (Highest)	Individual Employee Roster	Specific date shift override assigned to employee.	Applies individual roster schedule; terminates search.
Level 2	Department Default Shift	Employee belongs to department with active shift policy.	Applies department shift schedule (e.g. Support 24/7).
Level 3	Branch / Site Shift	Branch facility operates on localized operational hours.	Applies site-specific shift timing.
Level 4 (Lowest)	Company Standard Default	No custom overrides configured in hierarchy.	Falls back to standard (09:30 AM to 06:30 PM).

Overnight Shifts (Crossing Midnight): In 24/7 operational facilities (e.g. 21:00 PM to 06:00 AM next morning), treating calendar midnight as a day boundary causes critical calculation errors: check-in occurs on Day 1, while check-out occurs on Day 2. The shift engine resolves this by anchoring all punches within a rolling 24-hour window to the *Shift Commencement Date*. A check-out at 06:00 AM on September 19th is automatically attributed to the shift commencing on September 18th.

14.6 Shared Mutation Validation & Bulk Marking

To prevent inconsistent writes across manual single edits, bulk spreadsheet imports, and administrative overrides, the system funnels all writes through `IAttendanceMutationValidator`. This validator guarantees that every write enforces identical business rules:

- Target month must NOT be locked (`IsLocked == false`).
- Employee must have been active on target date (`JoiningDate <= Date <= ExitDate`).
- Date cannot be a configured company holiday unless holiday-shift override is active.
- Leave-owned or WFH-owned rows must NOT be overwritten silently (treated as protected skips).

14.7 Chapter 14 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-14-001`	CRITICAL	Manual bulk attendance marking overwritten approved leave rows with "Present" without warning, corrupting employee leave balances and payroll LOP calculations.	Implement protected-skip pattern in `AttendanceBulkMutationService`: check `SourceType != MANUAL`; if source-linked, skip write and add to `ProtectedSkips` response summary.
`DEF-14-002`	HIGH	Employee check-in without check-out on same day left `TotalHours = null`, causing monthly validation to crash with unhandled `NullReferenceException`.	Catch missing check-outs during daily aggregation; flag record with `IsMissingPunch = true` and assign provisional status requiring HR reconciliation.
`DEF-14-003`	HIGH	Overnight night shift workers (10:00 PM to 06:00 AM) were marked absent on day 1 and early exit on day 2 due to midnight date splitting.	Implement shift anchor allocation: punches within shift buffer are assigned to shift origin date; total hours computed across midnight boundary.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-14-01`	Shift: 09:30 - 18:30 (Grace: 15 min).	Employee clocks in at 09:46 AM.	Record saved with `LateCount = 1`; flagged for monthly penalty review.
`QA-14-02`	Payroll month August 2026 is locked.	Admin attempts manual edit on 2026-08-10.	HTTP 400 Bad Request; "Attendance period is locked against modifications".
`QA-14-03`	Biometric device sends duplicate punch within 30 seconds.	Device pushes duplicate record to ingestion webhook.	Duplicate discarded; only first punch ingested into `AttendanceModel`.

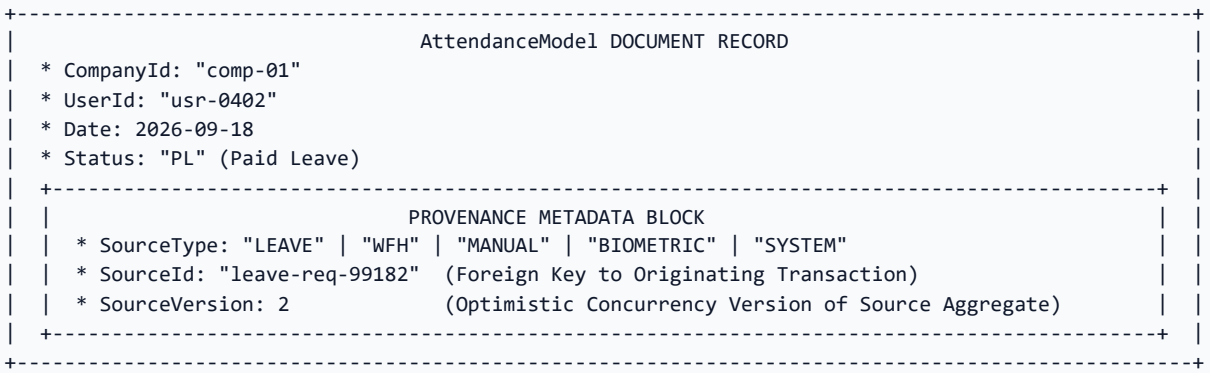
15. Source-Linked Attendance Reconciliation Engine

15.1 Provenance Architecture & Data Integrity Challenge

The most pervasive architectural defect in enterprise HRMS platforms is *attendance data corruption through uncoordinated writes*. When an employee takes approved Sick Leave, the Leave module writes a leave record. Subsequently, an HR clerk runs a bulk attendance import marking all departmental workers as "Present", or an employee accidentally clocks in. Without provenance tracking, the approved leave is silently overwritten, invalidating leave balances and corrupting payroll calculations.

The Codeji HRIS eliminates this architectural defect through **Source-Linked Provenance Tracking**. Every attendance record maintains explicit ownership metadata:

FIGURE 15.1: SOURCE PROVENANCE METADATA & RECONCILIATION DATA MODEL



15.2 Reconciliation State Transition Invariants

The reconciliation engine enforces a strict state matrix governing write conflicts:

Existing Row Source	Incoming Operation	System Reconciliation Action	Resulting Record State
`LEAVE (Approved)`	Bulk Mark "Present"	**Protected Skip:** Bypass write; preserve leave row.	Unchanged; added to bulk response ProtectedSkips list.
`WFH (Approved)`	Manual HR Punch Edit	**Protected Skip:** Rejects uncoordinated overwrite.	HTTP 409 Conflict; requires explicit WFH cancellation first.
`BIOMETRIC (Clock)`	Approved Half-Day Leave	**Source Merge:** Reconciles punch duration with leave.	Status = "HD + PL"; SourceType updated to LEAVE.
`LEAVE (Approved)`	Leave Revocation / Cancellation	**Atomic Reversion:** Reverts status to default.	Status = "A" (or punch evaluated); Source cleared.

15.3 Deep-Dive Source Reconciliation Edge Cases

Real-world workforce operations generate complex cross-module edge cases that the reconciliation engine resolves deterministically:

- **Edge Case 1: Half-Day Approved Leave + Physical Biometric Punch:** An employee applies for first-half Casual Leave (09:30 AM to 01:30 PM) and arrives at the office at 01:45 PM for the second half. The biometric clock registers check-in

at 01:45 PM and check-out at 06:35 PM (Total: 4.83 hours). The engine detects that `TotalHours >= Shift.HalfDayThresholdHours` (4.5h), merges the two sources, and records: `Status = "Present (0.5) + CL (0.5)"` with payable weight = 1.0 day.

- **Edge Case 2: Approved WFH + Physical Check-In at Office:** An employee has an approved WFH day but comes into the office for a client meeting, scanning their biometric card. The reconciliation engine recognizes physical presence at the headquarters, updates the row note: "*On-premise punch during approved WFH*", but preserves WFH quota consumption unless explicitly cancelled.
- **Edge Case 3: Weekend / Public Holiday Punch (Comp-Off Generation):** If an employee clocks >= 6.0 hours on a configured Weekly Off or Company Holiday, the daily aggregator generates a provisional `CompOffGrantRecord` awaiting manager verification. Upon manager approval, the employee's compensatory leave balance is credited with 1.0 day.

15.4 Chapter 15 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-15-001`	CRITICAL	Leave cancellation threw unhandled exception when attendance record had already been edited by HR, leaving leave balance un-refunded.	Use conditional update in reconciliation service: if `SourceId` matches, revert status; if `SourceId` mismatched (manual override), refund leave balance and log reconciliation audit alert.
`DEF-15-002`	HIGH	Frontend attendance grid rendered generic "Absent" icon for approved leave days because API omitted `SourceType` in response DTO.	Update `AttendanceResponseDto` to expose `SourceType` and `SourceId`; frontend displays distinct Leave/WFH badge with tooltip.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-15-01`	Attendance row has `SourceType = "LEAVE"`.	Dispatch bulk attendance mark "Present" for company.	Row remains untouched; bulk mark response includes employee in `ProtectedSkips`.
`QA-15-02`	Approved leave cancelled by HR.	Execute leave cancellation workflow.	Attendance row reverted; leave balance incremented by exact leave days.

16. Attendance Penalty Engine, Exception Auditing & Monthly Lock Lifecycle

16.1 LHD/ED Penalty Engine & Calculation Formulas

To enforce workplace punctuality and compliance with agreed shift timings, the system incorporates a configurable Late Arrival & Early Departure (LHD/ED) Penalty Engine. Punctuality infractions accumulate over a calendar month and translate mathematically into Loss of Pay (LOP) deductions:

MATHEMATICAL FORMULATION: LHD/ED ATTENDANCE PENALTY RULES

```
Let MonthlyLateCount = SUM(DailyAttendance.LateCount) for Month M
Let MonthlyEarlyExitCount = SUM(DailyAttendance.EarlyExitCount) for Month M
Let TotalInfractions = MonthlyLateCount + MonthlyEarlyExitCount

Standard Enterprise Policy Tiering:
- If TotalInfractions <= 3 ----> PenaltyLOPDays = 0.0 (Grace Threshold)
- If TotalInfractions == 4 ----> PenaltyLOPDays = 0.5 Day Loss of Pay
- If TotalInfractions == 5 ----> PenaltyLOPDays = 1.0 Day Loss of Pay
- If TotalInfractions > 5 ----> PenaltyLOPDays = 1.0 + (TotalInfractions - 5) * 0.5 Days LOP

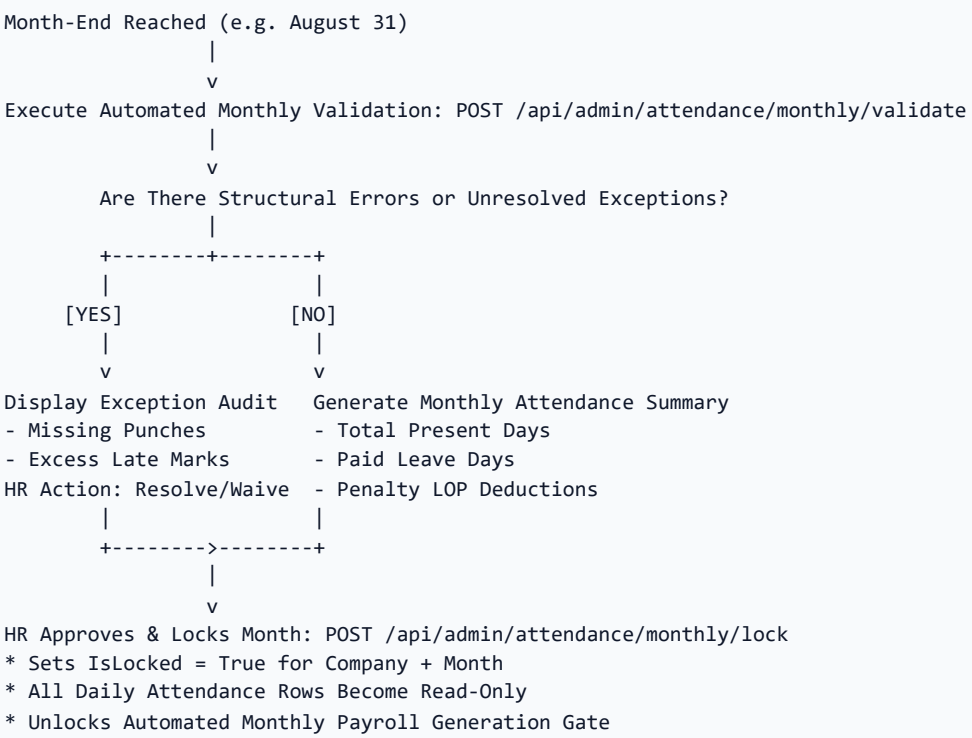
NetPayableDays = TotalCalendarDaysInMonth - UnpaidAbsences - ApprovedUnpaidLeaves - PenaltyLOPDays
```

16.2 Monthly Exception Auditing & Resolution Workflow

Before any month's attendance can be finalized for payroll computation, the system executes an automated structural audit to identify exceptions:

- **Missing Punch Exceptions:** Single check-in without check-out, or check-out without check-in.
- **Invalid Duration Exceptions:** Total hours exceeding 18 hours or negative durations caused by improper clock times.
- **Unresolved Penalty Exceptions:** Accumulated LHD infractions exceeding the grace threshold awaiting HR waiver or deduction confirmation.

FIGURE 16.2: MONTHLY ATTENDANCE AUDITING & PAYROLL LOCKING PIPELINE



16.3 Step-by-Step Monthly Attendance Locking Runbook

Operations teams follow a mandatory 4-step protocol at month-end to prepare attendance for payroll:

1. **Step 1: Automated Validation Trigger:** Dispatch `POST /api/admin/attendance/monthly/validate?month=8&year=2026`. The system scans all attendance rows for the month and returns an itemized exception manifest.
2. **Step 2: Structural Exception Remediation:** For missing punches, HR contacts employees to confirm actual departure times and performs manual regularizations via the Exception Grid.
3. **Step 3: Policy Penalty Waivers:** HR reviews employees with 4+ late marks; discretionary medical or operational waivers are recorded (`IsPenaltyWaived = true`).
4. **Step 4: Formal Month Locking:** HR clicks *Approve & Lock Attendance*. The system asserts `StructuralExceptionsCount == 0`, updates `IsLocked = true`, and issues a cryptographic lock timestamp.

16.4 MongoDB Exception Manifest & Audit Aggregation Queries

// 1. Audit Query: Find Unresolved Missing Punches for Company in Given Month db.Attendance.aggregate([{ $match: { CompanyId: "comp-01", Date: { $gte: ISODate("2026-09-01T00:00:00Z"), $lt: ISODate("2026-10-01T00:00:00Z") }, $or: [{ CheckInTime: { $ne: null }, CheckOutTime: null }, { CheckInTime: null, CheckOutTime: { $ne: null } }], Status: { $in: ["Present", "HalfDay"] } } }, { $project: { UserId: 1, Date: 1, CheckInTime: 1, CheckOutTime: 1, SourceType: 1 } }]); // 2. Audit Query: Aggregate Monthly Late Counts & Identify Employees Exceeding Grace Threshold db.Attendance.aggregate([{ $match: { CompanyId: "comp-01", Date: { $gte: ISODate("2026-09-01T00:00:00Z"), $lt: ISODate("2026-10-01T00:00:00Z") }, LateCount: { $gt: 0 } } }, { $group: { _id: "$UserId", TotalLateMarks: { $sum: "$LateCount" }, WaivedMarks: { $sum: { $cond: ["$IsPenaltyWaived", 1, 0] } } } }, { $project: { UserId: "$_id", TotalLateMarks: 1, EffectiveLateMarks: { $subtract: ["$TotalLateMarks", "$WaivedMarks"] } } }, { $match: { EffectiveLateMarks: { $gt: 3 } } }]);

16.5 Emergency Unlock & Rollback Governance Matrix

Governance State	Permissible Action	Required Authorization	System Audit & Invariant Enforcement
Attendance Locked, Payroll Not Started	Emergency Attendance Unlock	HR Director Approval	Audit log records unlock justification; month transitions back to `IsLocked = false`.
Attendance Locked, Payroll in Draft	Conditional Unlock with Draft Invalidation	Finance Controller + HR Director	Existing draft payroll records automatically invalidated and purged; forces re-computation.
Attendance Locked, Payroll Published/Disbursed	**Strictly Forbidden**	Blocked by Architecture	HTTP 409 Conflict. Cannot unlock attendance after salary disbursement. Requires arrears in next month.

16.6 Chapter 16 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-16-001`	CRITICAL	HR was able to unlock a previously locked attendance month even after payroll had been processed and paid out, desynchronizing payslips from attendance records.	Enforce immutable payroll dependency check: monthly attendance unlock is strictly forbidden if `MonthlyPayroll.IsProcessed == true` for that company/month.
`DEF-16-002`	HIGH	Late marks forgiven/waived by HR in the exception audit screen were re-applied by background recalculation jobs.	Persist explicit `IsPenaltyWaived = true` and `WaivedByUserId` flags on the daily attendance document; recalculation engine respects waived flags.
`DEF-16-003`	HIGH	Monthly validation job timed out when processing company with 1,000+ employees due to N+1 database queries per employee date.	Refactor monthly validation to execute a single bulk MongoDB aggregation pipeline group query across the entire month.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-16-01`	Employee has 4 late marks in month.	Run monthly attendance summary calculation.	Summary records `PenaltyLOPDays = 0.5`; net payable days reduced accordingly.
`QA-16-02`	Unresolved missing punch exception exists on 2026-08-20.	Attempt to invoke `POST /api/admin/attendance/monthly/lock`.	HTTP 400 Bad Request; "Cannot lock attendance with unresolved structural exceptions".
`QA-16-03`	Payroll processed and published for August 2026.	Admin attempts to unlock August attendance.	HTTP 409 Conflict; "Cannot unlock attendance: payroll already processed and published".

PART V: LEAVE MANAGEMENT & COMPENSATION SYSTEMS

Leave Policies, Accruals, Salary Architecture, Divisors & Payroll Engine

Authoritative specification of leave accrual algorithms, sandwich rule engines, salary component structuring, Section 115BAC taxation regimes, 4-tier payroll divisor frameworks, 5 concrete employee payroll arithmetic walkthroughs, draft vs publish gates, bank disbursement advice, and automated payslip PDF generation.

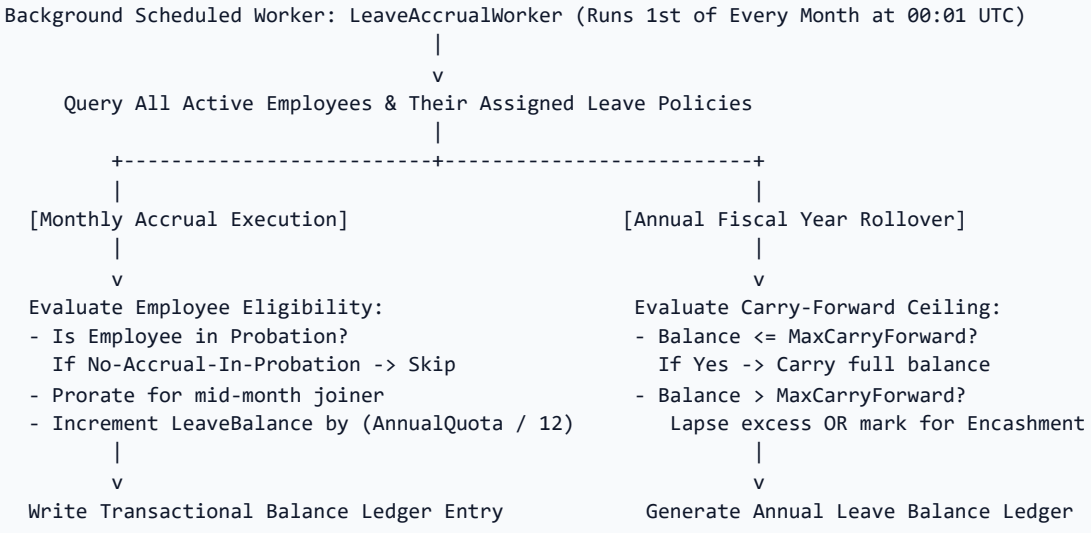
17. Leave Policy Framework & Background Accrual System

17.1 Business Drivers, Regulatory Mandate & Architecture

Employee leave management is subject to strict statutory labor laws (Factories Act, Shops & Establishments Act). Organizations must maintain accurate records of earned leave entitlements, casual allowances, sick leave with medical evidence, and maternity/paternity protections. Failure to accurately compute leave accruals or carry-forward balances results in employee dissatisfaction, labor audit violations, and erroneous financial liability accruals on corporate balance sheets.

The Codeji Leave Policy Framework provides a fully automated policy definition and balance management engine. Company administrators define distinct policies covering accrual rates, probationary waiting periods, maximum accumulation ceilings, and negative balance permissions.

FIGURE 17.1: LEAVE ACCRUAL & ANNUAL BALANCE ROLLOVER PIPELINE



17.2 Statutory Leave Entitlements Across State Jurisdictions

Different states enforce distinct statutory minimum leave entitlements under local Shops & Commercial Establishments Acts:

State Jurisdiction	Statutory Privilege Leave (PL)	Casual / Sick Leave (CL / SL)	Max Accumulation Carry-Forward
Maharashtra	1 day for every 20 days worked (18 days/yr)	8 days Casual Leave	Maximum 45 days accumulation.
Karnataka	1 day for every 20 days worked (15 days/yr)	12 days Casual / Sick Leave	Maximum 30 days accumulation.
Delhi	15 days Privilege Leave	12 days Casual / Sick Leave	Maximum 45 days accumulation.
Tamil Nadu	12 days Privilege Leave	12 days Casual / Sick Leave	Maximum 45 days accumulation.

17.3 Multi-Tier Accrual Frequencies & Probation Waiting Periods

Organizations configure accrual intervals matching operational requirements:

- **Monthly Accrual (Standard):** Credit added on 1st of each calendar month (e.g. 1.5 days PL per month).
- **Quarterly Accrual:** Credit added at the beginning of each fiscal quarter (4.5 days every 3 months).
- **Annual Frontloaded Grant:** Entire annual entitlement (e.g. 12 Casual Leaves) credited on January 1st or April 1st.
- **Probationary Waiting Period:** Employees earn leave accruals during probation, but balance is marked `LockedInProbation` until formal confirmation, preventing absence abuse prior to tenure validation.

17.4 Mathematical Formulation & Accrual Rules

MATHEMATICAL FORMULATION: MONTHLY PRORATED LEAVE ACCRUAL

```
Let AnnualEntitlement = Policy.AnnualQuota (e.g. 18.0 Days for Privilege Leave)
Standard Monthly Credit = AnnualEntitlement / 12.0 = 1.50 Days per Month

Mid-Month Joiner Proration Formula:
If JoiningDate occurs in Month M on Day D:
Let DaysInMonth = DateTime.DaysInMonth(Year, Month)
Let ActiveDays = DaysInMonth - D + 1
ProratedCredit = (ActiveDays / DaysInMonth) * (AnnualEntitlement / 12.0)
Rounded to nearest half-day (0.5) or exact 2 decimal places per policy setting.

Annual Rollover & Lapsing Formula:
Let CurrentBalance = LeaveBalance.AvailableDays
Let MaxCarryForward = Policy.MaxCarryForwardLimit (e.g. 30.0 Days)
If CurrentBalance > MaxCarryForward:
- RetainedBalance = MaxCarryForward
- LapsedDays = CurrentBalance - MaxCarryForward (Optionally converted to EncashmentPayable)
```

17.5 Leave Policy Domain Models & C# Entity Definitions

// Leave Policy Aggregate Definition in CMS-Backend-Core [BsonCollection("LeavePolicies")] public class LeavePolicy : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string PolicyId { get; set; } = string.Empty; public string CompanyId { get; set; } = string.Empty; public string LeaveTypeCode { get; set; } = string.Empty; // "PL", "CL", "SL", "COMP_OFF" public string Description { get; set; } = string.Empty; public decimal AnnualQuota { get; set; } = 18.0m; public AccrualFrequency Frequency { get; set; } = AccrualFrequency.Monthly; public bool AccrueDuringProbation { get; set; } = false; public bool AllowEncashment { get; set; } = true; public decimal MaxCarryForwardDays { get; set; } = 30.0m; public bool EnableSandwichRule { get; set; } = true; public bool AllowNegativeBalance { get; set; } = false; public decimal MaxNegativeDaysLimit { get; set; } = 0.0m; } // Transactional Leave Balance Ledger Document [BsonCollection("LeaveBalances")] public class LeaveBalanceModel : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string BalanceId { get; set; } = string.Empty; public string

CompanyId { get; set; } = string.Empty; public string UserId { get; set; } = string.Empty; public string LeaveTypeCode { get; set; } = string.Empty; public int FiscalYear { get; set; } = 2026; public decimal OpeningBalance { get; set; } = 0.0m; public decimal AccruedDays { get; set; } = 0.0m; public decimal ConsumedDays { get; set; } = 0.0m; public decimal EncashedDays { get; set; } = 0.0m; public decimal LapsedDays { get; set; } = 0.0m; public decimal AvailableBalance => OpeningBalance + AccruedDays - ConsumedDays - EncashedDays - LapsedDays; }

17.6 Chapter 17 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-17-001`	CRITICAL	Monthly accrual worker executing on 1st of month failed midway due to transient MongoDB timeout, causing partial accruals without transactional recovery.	Wrap monthly accrual loop in transactional batching with idempotency ledger: records written with `AccrualBatchKey = "{CompanyId}_{Year}_{Month}"`. Retries skip already-processed employees.
`DEF-17-002`	HIGH	Employees resigned mid-year were credited full annual leave front-load, leading to overpayment in final settlement leave encashment.	Enforce exit-date proration: when employee separation date is confirmed, execute dynamic claw-back on unearned front-loaded leave balance.

18. Leave Transaction Processing & Sandwich Rule Mechanics

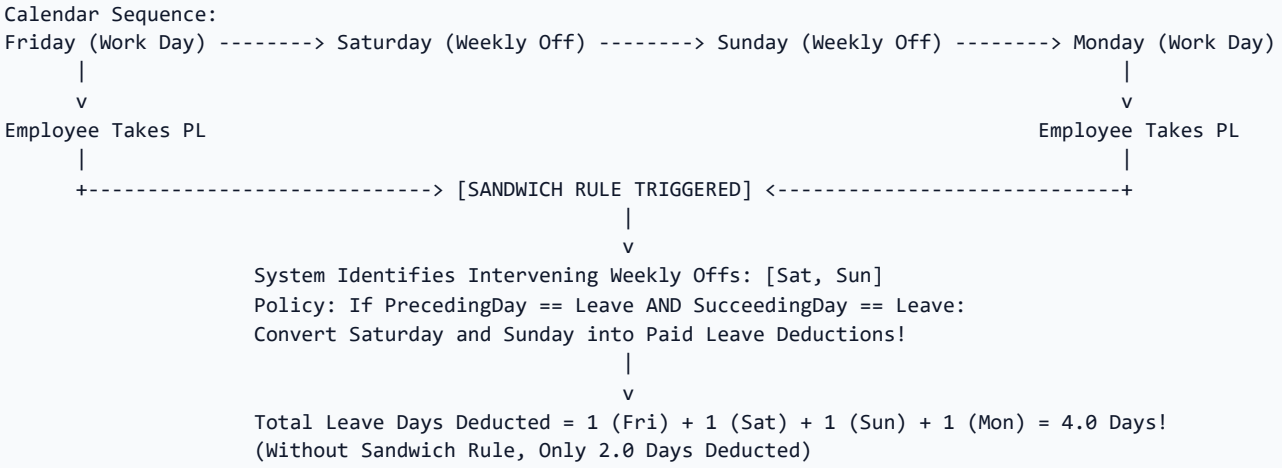
18.1 Leave Application Lifecycle & Manager Approval Workflows

Employees request leaves through self-service specifying LeaveType, StartDate, EndDate, DayPortion (FullDay / FirstHalf / SecondHalf), and Reason. Upon submission, the engine checks balance availability, validates sandwich rules, and creates a pending review item for the reporting manager.

18.2 Sandwich Rule Engine: Weekend & Holiday Penalization

The Sandwich Rule is an enterprise attendance discipline policy designed to discourage employees from artificially extending weekends or gazetted holidays. If an employee takes leave on the working day immediately preceding AND following a non-working period, the intervening non-working days are automatically converted into deducted leaves:

FIGURE 18.1: SANDWICH RULE INTERVENING OFF-DAY PENALIZATION ENGINE



18.3 Algorithmic Detection & Cross-Month Sandwich Rule Handling

The Sandwich Rule Engine scans across boundary dates using a lookahead and lookbehind window:

- **Weekend Sandwich:** Friday Leave + Monday Leave → Saturday + Sunday converted to leave.
- **Gazetted Holiday Sandwich:** Leave on day before national holiday (e.g. August 14th) + Leave on day after (August 16th) → August 15th holiday converted to leave deduction.
- **Cross-Month Boundary Sandwich:** If an employee takes leave on the last working day of August (Friday August 31st) and the first working day of September (Monday September 3rd), the engine links across the monthly payroll boundary, applying the weekend deductions to the August or September ledger per policy rules.

18.4 Chapter 18 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-18-001`	HIGH	Sandwich rule penalized employees when Monday was a company holiday followed by work on Tuesday, misclassifying holiday as leave.	Enforce calendar-aware terminal condition: sandwich evaluation only triggers if the succeeding work day is explicitly marked as Leave, skipping intermediate public holidays.
`DEF-18-002`	MEDIUM	Leave cancellation did not restore intervening weekend sandwich deductions, leaving balance short by 2 days.	Store parent transaction ID on sandwich deduction rows; cancelling the leave application cascades to remove all linked sandwich deduction rows.

19. Salary Structure Architecture & Component Engineering

19.1 CTC Decomposition & Component Taxonomy

In enterprise compensation systems, Cost to Company (CTC) is decomposed into fixed earnings, variable incentives, statutory employer contributions, and tax withholdings. Every component is defined with exact calculation formulas, tax treatment, and attendance dependency:

Salary Component	Classification	Mathematical Formulation	Statutory / Tax Treatment
Basic Salary	Earning (Fixed)	40% to 50% of Total Monthly Gross	Mandatory base for Provident Fund & Gratuity calculations.
House Rent Allowance (HRA)	Earning (Taxable)	40% (Non-Metro) or 50% (Metro) of Basic Salary	Eligible for Section 10(13A) tax exemption upon rent proof.
Special Allowance	Earning (Taxable)	Balancing component: `CTC - (Basic + HRA + Allowances)`	Fully taxable allowance.
Employee Provident Fund (EPF)	Statutory Deduction	12% of Basic Salary (Statutory cap: ₹1,800/mo)	Employees' Provident Funds Act, 1952.
Employee State Insurance (ESIC)	Statutory Deduction	0.75% of Gross Wages (Employer: 3.25%)	Applicable if monthly gross wages <= ₹21,000.
Professional Tax (PT)	Statutory Deduction	State-specific slab (e.g. ₹200/month)	State Tax on Professions and Trades.

19.2 Indian Income Tax Slabs (Old vs New Tax Regime Under Section 115BAC)

The payroll tax calculation engine evaluates income tax withholding (TDS) under Section 192 under both taxation regimes:

Annual Taxable Income Slab	New Tax Regime (Section 115BAC Default)	Old Tax Regime (With Chapter VI-A Exemptions)
Up to ₹3,00,000	Nil (0%)	Nil (Up to ₹2,50,000)
₹3,00,001 - ₹6,00,000	5% (Full Rebate u/s 87A up to ₹7.0 Lakhs)	5% (₹2,50,001 - ₹5,00,000)
₹6,00,001 - ₹9,00,000	10%	20% (₹5,00,001 - ₹10,00,000)
₹9,00,001 - ₹12,00,000	15%	20%
₹12,00,001 - ₹15,00,000	20%	30% (Above ₹10,00,000)
Above ₹15,00,000	30%	30%

Tax Regime Exemption Comparison:

- **New Tax Regime (Default):** Standard Deduction of ₹75,000 (FY 2024-25 onwards); Section 87A rebate eliminates tax liability for net taxable income up to ₹7,00,000. No Chapter VI-A deductions (80C, 80D, HRA) allowed.
- **Old Tax Regime:** Standard Deduction of ₹50,000; Section 80C deductions up to ₹1,50,000; Section 80D medical insurance up to ₹25,000 (₹50,000 for senior parents); Section 24 housing loan interest up to ₹2,00,000; Section 10(13A) HRA exemption based on actual rent receipts.

19.3 State-Wise Professional Tax (PT) Statutory Schedules

State Jurisdiction	Monthly Gross Wage Slab	Applicable Monthly PT	Special Rules
Maharashtra	Men: > ₹10,000 / mo; Women: > ₹25,000 / mo	₹200 / month	₹300 in February (₹2,500 annual max).
Karnataka	Gross >= ₹15,000 / month	₹200 / month	Nil if gross < ₹15,000.
Telangana	₹15,001 - ₹20,000: ₹150; > ₹20,000: ₹200	₹200 / month	Graduated progressive slab.
West Bengal	₹25,001 - ₹40,000: ₹150; > ₹40,000: ₹200	₹200 / month	Annual ceiling ₹2,500.
Tamil Nadu	Half-yearly slab: > ₹75,000 half-year	₹208 / month average	Deducted bi-annually (Sep & Feb).

19.4 The 4-Tier Payroll Divisor Engine

MATHEMATICAL FORMULATION: 4-TIER PAYROLL DIVISOR POLICIES

```
Let MonthlyGross = Fixed Monthly Gross Salary

Policy 1: Calendar Days Divisor (Default)
PerDayRate = MonthlyGross / DateTime.DaysInMonth(Year, Month)
(e.g., February = 28/29 days, August = 31 days)

Policy 2: Fixed 30 Days Divisor
PerDayRate = MonthlyGross / 30.0
(Standardized banking divisor regardless of calendar length)

Policy 3: Statutory 26 Days Divisor
PerDayRate = MonthlyGross / 26.0
(Excludes 4 standard weekly offs; standard under Indian Payment of Wages Act)

Policy 4: Working Days Divisor
PerDayRate = MonthlyGross / (CalendarDays - ConfiguredWeeklyOffs - GazettedHolidays)
(e.g., 21 or 22 actual working days in that specific month)
```

19.5 Chapter 19 Architectural Defect Audit & QA Matrix

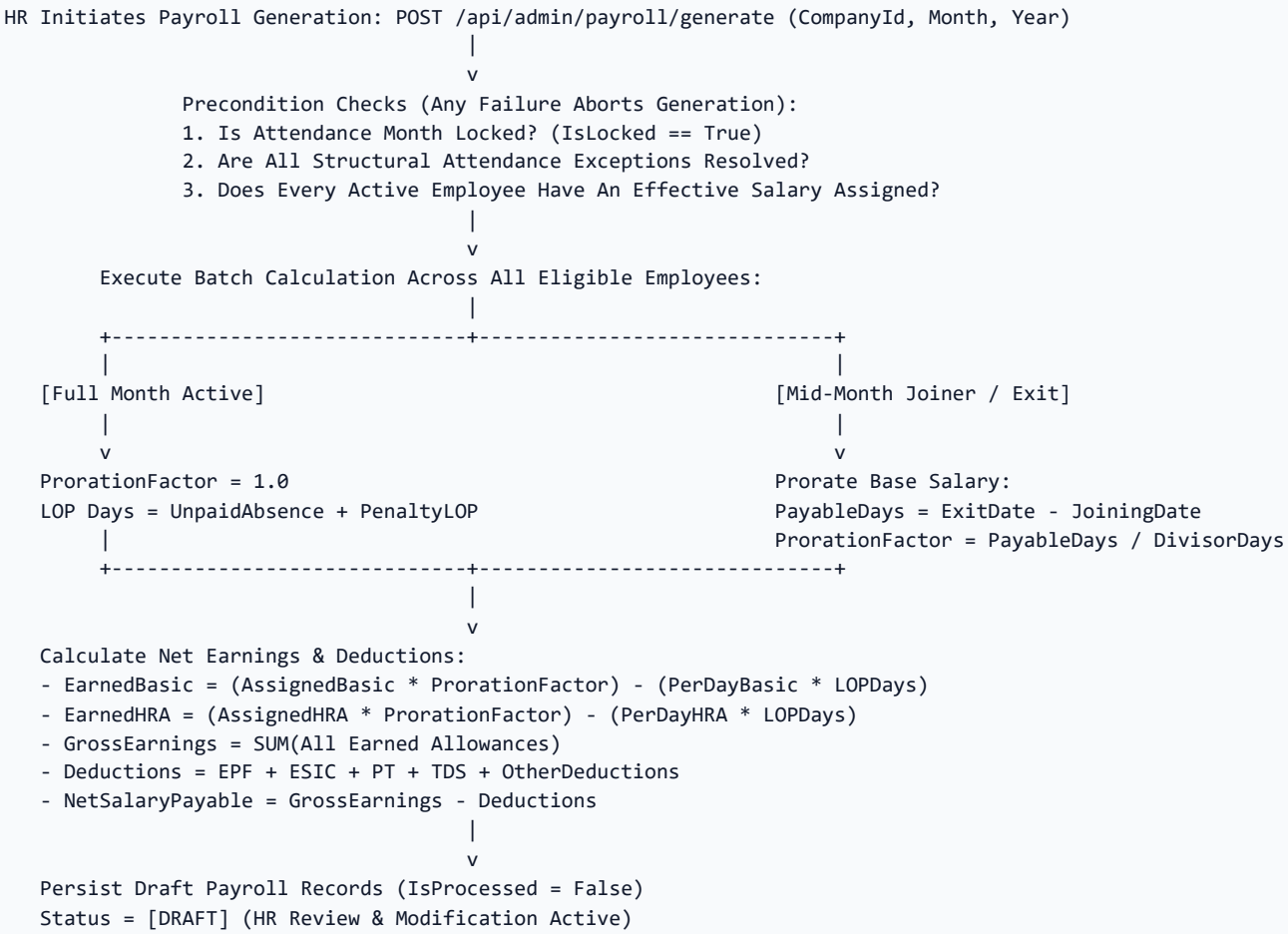
Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-19-001`	HIGH	PF deduction was calculated on entire gross salary rather than Basic Salary, resulting in excess statutory tax withholding.	Enforce statutory capping and component-specific targeting: EPF formula explicitly targets `SalaryStructure.Basic` with optional ₹15,000 statutory wage ceiling cap.
`DEF-19-002`	MEDIUM	In leap years, February divisor defaulted to 28 days instead of 29 days when using Calendar Days divisor policy.	Use `DateTime.DaysInMonth(year, month)` dynamic BCL method rather than static lookup arrays.
`DEF-19-003`	HIGH	Maharashtra Professional Tax deducted ₹200 in February instead of statutory ₹300, leading to non-compliance notice during annual PT assessment.	Implement calendar-month trigger in PT calculation engine: if `Month == 2` and `State == "Maharashtra"`, enforce ₹300 deduction.

20. Automated Monthly Payroll Generation, Proration & Divisor Processing

20.1 Payroll Generation Pipeline & Prerequisites

Automated Monthly Payroll Generation is the critical computational apex of the HRIS. It synthesizes employee master records, effective salary assignments, locked attendance summaries, leave loss-of-pay days, and penalty infractions into concrete monthly payroll line items.

FIGURE 20.1: AUTOMATED PAYROLL GENERATION & CALCULATION FLOW



20.2 Core Payroll Domain Entities & C# Model Definitions

// Monthly Payroll Aggregate Record in CMS-Backend-Core [BsonCollection("MonthlyPayroll")] public class MonthlyPayroll : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string PayrollId { get; set; } = string.Empty; public string CompanyId { get; set; } = string.Empty; public string UserId { get; set; } = string.Empty; public int Month { get; set; } public int Year { get; set; } // Attendance Parameters public int TotalCalendarDays { get; set; } public decimal PayableDays { get; set; } public decimal LossOfPayDays { get; set; } public decimal PenaltyLopDays { get; set; } // Earnings Ledger public decimal BasicEarned { get; set; } public decimal HraEarned { get; set; } public decimal SpecialAllowanceEarned { get; set; } public decimal BonusOrArrears { get; set; } public decimal GrossEarnings { get; set; } // Deductions Ledger public decimal EpfDeduction { get; set; } public decimal EsicDeduction { get; set; } public decimal ProfessionalTax { get; set; } public decimal IncomeTaxTds { get; set; } public decimal OtherDeductions { get; set; } public decimal TotalDeductions => EpfDeduction + EsicDeduction + ProfessionalTax + IncomeTaxTds + OtherDeductions; // Net Result public decimal NetSalaryPayable => GrossEarnings

- TotalDeductions; // Lifecycle State public bool IsProcessed { get; set; } = false; // False = Draft, True = Published public DateTime? ProcessedAt { get; set; } public string? ProcessedByUserId { get; set; } }

20.3 Five Concrete Employee Payroll Calculation Walkthroughs

The following five comprehensive test cases illustrate exact itemized rupee arithmetic computed by the payroll engine in a 30-day month (September 2026):

CASE 1: STANDARD CONFIRMED EMPLOYEE (₹1,20,000 MONTHLY CTC, 0 LOP, NEW REGIME)

CTC: ₹1,20,000/month. Basic: ₹60,000 (50%), HRA: ₹30,000 (25%), Special: ₹30,000 (25%).
Attendance: 30 Calendar Days, 0 Absent, 0 Penalty. Payable Days = 30.0.

Gross Earnings: Basic (₹60,000) + HRA (₹30,000) + Special (₹30,000) = ₹1,20,000.00
Deductions:
* EPF (Capped at ₹15,000 ceiling): 12% of ₹15,000 = ₹1,800.00
* ESIC: Gross > ₹21,000 → Exempt = ₹0.00
* PT (Karnataka): ₹200.00
* TDS (New Regime, Annual Gross ₹14.4L - ₹75k Std Ded = ₹13.65L Taxable → Annual Tax ₹1,13,000 / 12) = ₹9,417.00
Total Deductions = ₹1,800 + ₹0 + ₹200 + ₹9,417 = ₹11,417.00
Net Salary Payable = ₹1,20,000 - ₹11,417 = ₹1,08,583.00

CASE 2: MID-MONTH JOINER (₹80,000 MONTHLY CTC, JOINED SEP 16TH, CALENDAR DIVISOR)

Joined: September 16, 2026. Calendar Days = 30. Active Days = (30 - 16) + 1 = 15 Days.
Proration Factor = 15.0 / 30.0 = 0.50 (50%).
Monthly Structure: Basic ₹40,000, HRA ₹20,000, Special ₹20,000.

Earned Gross Earnings:
* Basic: ₹40,000 * 0.50 = ₹20,000.00
* HRA: ₹20,000 * 0.50 = ₹10,000.00
* Special: ₹20,000 * 0.50 = ₹10,000.00
Total Earned Gross = ₹40,000.00
Deductions: EPF (₹1,800.00) + ESIC (₹0.00) + PT (₹200.00) + TDS (Prorated ₹1,500.00) = ₹3,500.00
Net Salary Payable = ₹40,000 - ₹3,500 = ₹36,500.00

CASE 3: PENALTY & LOP DEDUCTIONS (₹60,000 CTC, 2 UNPAID LOP + 6 LATE MARKS)

Monthly Gross = ₹60,000. Divisor Policy = 30 Days. Per-Day Rate = ₹2,000.00.
Attendance Inputs: 2 Unpaid Leave Days + 6 Late Marks (Exceeds grace 3 by 3: 4th=0.5, 5th=0.5, 6th=0.5 → 1.5 Days Penalty LOP).
Total Deductible Days = 2.0 (Unpaid) + 1.5 (Penalty LOP) = 3.5 Days.
Payable Days = 30.0 - 3.5 = 26.5 Days.

Earned Gross: 26.5 * ₹2,000.00 = ₹53,000.00 (Basic ₹26,500, HRA ₹13,250, Special ₹13,250)
Deductions: EPF (₹1,800.00) + ESIC (₹0.00) + PT (₹200.00) + TDS (₹800.00) = ₹2,800.00
Net Salary Payable = ₹53,000 - ₹2,800 = ₹50,200.00

CASE 4: HIGH-INCOME EXECUTIVE (₹2,50,000 MONTHLY CTC, OLD REGIME DEDUCTIONS)

CTC: ₹2,50,000/mo = ₹30,00,000/yr. Basic: ₹1,25,000, HRA: ₹62,500, Special: ₹62,500.
Declared Exemptions: 80C (₹1,50,000) + 80D (₹50,000) + Sec 24 Home Loan (₹2,00,000) + HRA Rent Exemption (₹2,40,000).
Annual Taxable Income: ₹30,00,000 - ₹50,000 (Std Ded) - ₹6,40,000 (Total Exemptions) = ₹23,10,000.
Old Regime Tax: ₹1,12,500 (up to 10L) + 30% of ₹13,10,000 (₹3,93,000) = ₹5,05,500 + 4% Cess (₹20,220) = ₹5,25,720/yr.
Monthly TDS Withholding = ₹5,25,720 / 12 = ₹43,810.00

Deductions: EPF (₹1,800.00) + PT (₹200.00) + TDS (₹43,810.00) = ₹45,810.00
Net Salary Payable = ₹2,50,000 - ₹45,810 = ₹2,04,190.00

CASE 5: BLUE-COLLAR / ENTRY-LEVEL WORKER (₹18,500 GROSS WAGES, TRIGGERING ESIC)

```
Gross Wages = ₹18,500.00 (Below ₹21,000 statutory limit → ESIC Mandated). Basic: ₹11,100 (60%).
Attendance: 30 Calendar Days, 0 LOP. Payable Days = 30.0.

Deductions:
* Employee EPF: 12% of ₹11,100 = ₹1,332.00
* Employee ESIC: 0.75% of ₹18,500 = ₹138.75 → Rounded to ₹139.00
* PT (Karnataka: > ₹15,000) = ₹200.00
* TDS: Below taxable slab → ₹0.00
Total Deductions = ₹1,332 + ₹139 + ₹200 = ₹1,671.00
Net Salary Payable = ₹18,500 - ₹1,671 = ₹16,829.00
```

20.4 Chapter 20 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-20-001`	CRITICAL	Draft payroll generation ran successfully even when attendance month was unlocked, allowing payroll to be calculated on dirty, fluctuating attendance data.	Enforce strict precondition guard in `PayrollGenerationService`: assert `MonthlyAttendanceSummary.IsLocked == true`; abort immediately with HTTP 412 Precondition Failed if unlocked.
`DEF-20-002`	HIGH	Employees who joined on the 31st day of a 31-day month received 0 pay due to an off-by-one error in `(DaysInMonth - JoiningDay)` proration calculation.	Correct inclusive date math: `PayableDays = (DaysInMonth - JoiningDate.Day) + 1`.

21. Payroll Review, Publication Gate & Payslip PDF Generation Engine

21.1 Two-Phase Publication Gate & Employee Visibility Barrier

A fundamental security and operational requirement in payroll processing is the absolute separation between draft payroll calculations and published employee payslips. During the review window, HR administrators must inspect calculations, adjust one-time bonuses or travel reimbursements, and verify bank disbursement advice files without prematurely exposing draft numbers to employees.

FIGURE 21.1: TWO-PHASE PAYROLL PUBLICATION & PAYSLIP PDF PIPELINE

```
Draft Payroll Records Persisted (IsProcessed = False)
                        |
                        v
       HR Admin Reviews Draft Payroll Summary Grid:
       - Verify Total Gross Payroll & Company EPF/ESIC Outflow
       - Add One-Time Performance Bonuses or Custom Deductions
       - Export Bank Disbursement Advice (NEFT / RTGS Excel Format)
                        |
                        v
       Finance Director Signs Off & Authorizes Release
                        |
                        v
       Process Payroll Command: POST /api/admin/payroll/process
       * Sets IsProcessed = True for Company + Month
       * Dispatches Asynchronous PDF Payslip Pre-Generation Queue
       * Broadcasts SignalR Event: "PayrollPublished" to All Employees
                        |
                        v
       Employee Portal Self-Service Unlocked:
       - Employee Can Now View Salary Breakdown in Mobile / Web App
       - Download Cryptographically Verified PDF Payslip
```

21.2 Bank Disbursement Advice Generation (HDFC, ICICI & SBI Formats)

Upon payroll processing, the finance engine exports direct deposit batch files formatted for enterprise banking portals:

Column Header	Mapping Field	Data Validation Rule	Banking Clearing Purpose
`BeneficiaryName`	`PersonalDetails.FullName`	Max 40 chars; Alphabetic only	Account title verification at receiving bank.
`BeneficiaryAccount`	`BankDetails.AccountNumber`	Numeric; 9 to 18 digits	Target credit account.
`IFSCCode`	`BankDetails.IFSCCode`	11 characters alphanumeric	RBI clearing branch identification.
`Amount`	`MonthlyPayroll.NetSalaryPayable`	Positive decimal; 2 decimal places	Exact net disbursement amount.
`Narration`	`"SALARY FOR " + Month + " " + Year`	Max 30 characters	Transaction description on employee statement.

21.3 Corporate Banking File Layouts: HDFC E-CMS vs ICICI Eazypay

Different corporate banking portals enforce specific file structure layouts:

- **HDFC E-CMS (CSV Format):** `RecordType, BeneficiaryCode, AccountNo, Amount, BeneficiaryName, DraweeLocation, PrintLocation, BeneficiaryEmail, PaymentRef, PaymentDate`. Requires UTF-8 encoding with CRLF row delimiters.
- **ICICI Eazypay (Fixed-Width TXT):** Formatted with strict byte-offset columns: Col 1-15 (Debit A/C), Col 16-30 (Beneficiary A/C), Col 31-41 (IFSC Code), Col 42-56 (Amount in Paise padded with leading zeros), Col 57-96 (Beneficiary Name padded with spaces).
- **SBI Corporate Banking (Bulk Excel):** Requires Excel format (.xlsx) with pre-validated IFSC routing codes and checksum verification.

21.4 High-Fidelity Payslip PDF Generation Engine

The payslip PDF generator is powered by Playwright Headless Chromium, utilizing server-side CSS-paged rendering. Every generated payslip adheres to enterprise compliance standards:

- **Corporate Branding:** Company logo, registered legal entity name, corporate identification number (CIN), and registered address.
- **Employee Identifiers:** Legal Name, EmployeeId, Designation, Department, Date of Joining, Bank Account Number (masked: `XXXX-XXXX-1234`), IFSC Code, PAN, and UAN.
- **Attendance Summary:** Total Calendar Days, Days Worked, Paid Leave Days, Loss of Pay (LOP) Days, and Net Payable Days.
- **Itemized Ledger:** Side-by-side tabular layout of Earnings (Basic, HRA, Special Allowance) vs. Deductions (EPF, ESIC, PT, TDS).
- **Cryptographic Digital Signature:** Digital authenticity block with generation timestamp, authorized signatory name, and verification QR code.

21.5 Chapter 21 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-21-001`	CRITICAL	Employee was able to download payslip PDF before HR publication by guessing the direct URL `/api/payroll/download/{payrollId}`.	Enforce `IsProcessed == true` assertion inside the payslip generation and download handler; return HTTP 404 if draft.
`DEF-21-002`	MEDIUM	Payslip PDF generation queued on main HTTP thread caused request timeout under high concurrency when publishing payroll for 500+ employees.	Offload PDF rendering to background worker queue (`Channel<PayslipRenderJob>`) or render on-demand upon individual employee download request.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-21-01`	Payroll in Draft state (`IsProcessed = false`).	Employee requests `GET /api/payroll/me/slips`.	Draft month omitted from response list; zero draft records visible.
`QA-21-02`	HR executes `POST /api/admin/payroll/process`.	Verify database state of payroll records.	All records for company/month updated to `IsProcessed = true`; timestamp recorded.

PART VI: EMPLOYEE OFFBOARDING & SEPARATION

Resignation, Departmental Clearance, F&F Settlement & Former Employee Portal

Exhaustive architectural reference for the non-destructive separation aggregate (EmployeeExit), multi-departmental clearance workflows, asset recovery depreciation schedules, gratuity and leave encashment formulation, Statement of Final Account ledgers, exit interview sentiment analytics, and magic-link former employee portal security.

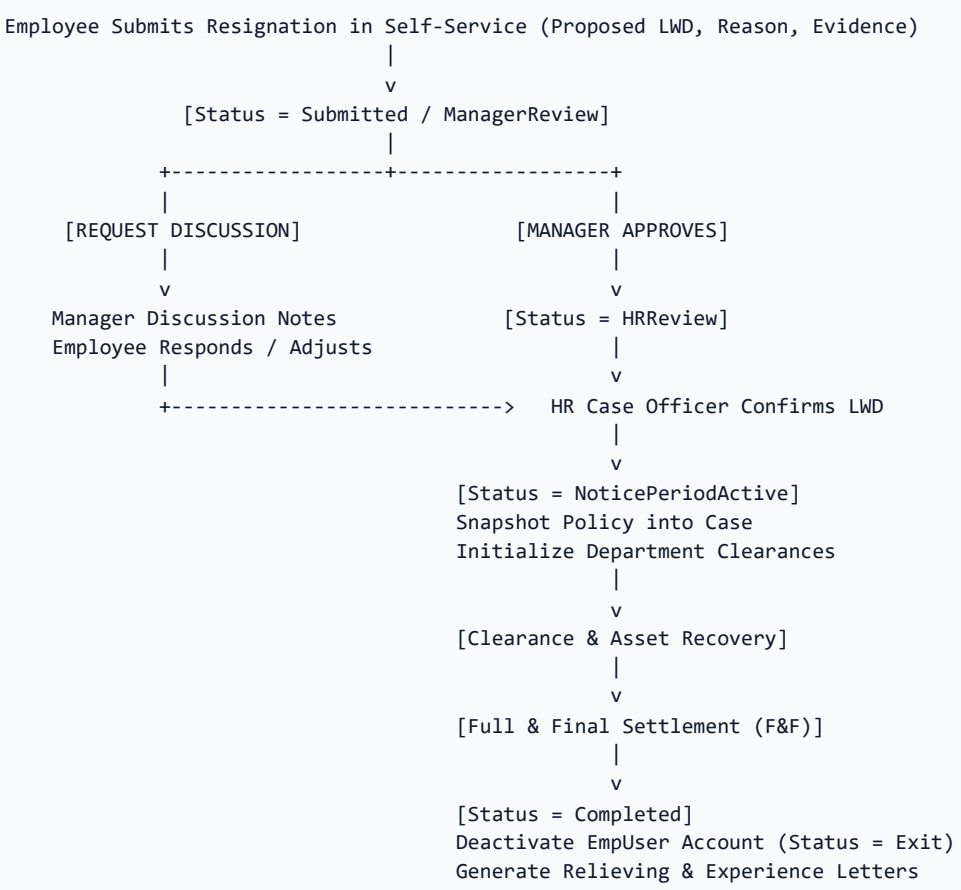
22. Employee Offboarding, Resignation Management & Notice Period Engine

22.1 Non-Destructive Separation Architecture & Core Drivers

Employee separation is among the most sensitive operations in enterprise human resources. Disorganized offboarding results in unrecovered company hardware, lingering access to sensitive corporate networks, statutory wage payment delays, and expensive labor disputes. A fundamental tenet of the Codeji HRIS is that **an exit never deletes an employee record**.

The system models separation through a dedicated aggregate root: `EmployeeExit`. Historical attendance, leave ledgers, performance reviews, and salary slips remain permanently archived in the primary database for multi-year statutory tax compliance and regulatory audits.

FIGURE 22.1: RESIGNATION & OFFBOARDING LIFECYCLE STATE MACHINE



22.2 Retention Discussion Journal & Counter-Offer Tracking

Prior to confirming an exit, the system facilitates retention efforts through a structured discussion protocol:

- **Manager Discussion Journal:** Captures primary turnover drivers (Compensation, Growth, Role Fit, Personal/Relocation, Management).
- **Counter-Offer Proposal:** Allows reporting managers to request salary adjustments or role re-assignment; automatically routes proposal to HR Business Partner and Finance Director.
- **Employee Resignation Withdrawal:** If retention discussion succeeds, employee submits formal withdrawal; upon mutual manager/HR approval, the exit case status transitions to `Withdrawn` and normal employment continues without interruption.

22.3 Notice Period Engine & Shortfall Buyout Formulation

The system calculates required notice days dynamically based on the employee's confirmation status and company policy snapshots (e.g. 30 days during probation, 90 days after confirmation). When an employee requests early departure, the system computes the notice shortfall buyout:

MATHEMATICAL FORMULATION: NOTICE PERIOD SHORTFALL & BUYOUT CALCULATION

```
Let RequiredNoticeDays = EmployeeExit.PolicySnapshot.NoticePeriodDays (e.g. 90 Days)
Let ActualServedDays = (ApprovedLWD - ResignationDate).TotalDays
Let ShortfallDays = MAX(0, RequiredNoticeDays - ActualServedDays)

NoticeBuyoutAmount = ShortfallDays * (Employee.MonthlyGrossSalary / 30.0)

If HR approves waiver -> NoticeBuyoutAmount = 0.0 (WaiverReason persisted in audit log)
```

22.4 Notice Buyout Tax Implications (Section 17(3))

Under Indian Income Tax rules, notice pay recovered from an employee is not deductible from taxable salary unless explicit company policy nets out the recovery prior to Form 16 issuance. Conversely, notice pay paid by the employer in lieu of notice is treated as *Profits in Lieu of Salary* under Section 17(3) and fully taxed at applicable slab rates.

22.5 Exit Interview Taxonomy & Attrition Sentiment Analytics

To identify systemic organizational friction points, departing employees complete an audited exit survey structured across 10 analytical dimensions:

Survey Dimension	Core Evaluation Question	Strategic HR Analytics Utilization
Compensation & Benefits	Competitiveness of base pay, health insurance, and annual bonuses.	Benchmarked against industry Radford / Mercer survey ranges.
Career Progression	Clarity of promotion pathways and internal mobility opportunities.	Identifies departments with high tenure ceilings and talent stagnation.
Managerial Effectiveness	Quality of supervision, psychological safety, and regular feedback.	Aggregated anonymously into Manager Leadership Index scorecards.
Work-Life Balance	Frequency of overtime, weekend work, and shift sustainability.	Correlated with historical daily attendance hours and LHD/ED metrics.
Organizational Culture	Alignment with corporate values, inclusion, and transparency.	Monitored for sudden sentiment drops indicating leadership misalignment.
eNPS (Net Promoter Score)	"How likely are you to recommend Codeji as an employer to peers?"	Tracked on a -100 to +100 scale as a core People Operations KPI.

22.6 Core Offboarding Domain Entities & C# Model Definitions

// Core Employee Exit Aggregate in CMS-Backend-Core [BsonCollection("EmployeeExit")] public class EmployeeExit : BaseEntity { [BsonId] [BsonRepresentation(BsonType.ObjectId)] public string ExitId { get; set; } = string.Empty; public string CompanyId { get; set; } = string.Empty; public string UserId { get; set; } = string.Empty; public DateTime ResignationDate { get; set; } public DateTime ProposedLwd { get; set; } public DateTime? ApprovedLwd { get; set; } public string ReasonCategory { get; set; } = string.Empty; public string DetailedReason { get; set; } = string.Empty; public ExitStatus Status { get; set; } = ExitStatus.Submitted; // Policy Snapshot at Exit Initiation public ExitPolicySnapshot PolicySnapshot { get; set; } = new(); // Department Clearances Checklist public List<DepartmentClearanceItem> Clearances { get; set; } = new(); // Settlement Ledger Record public FinalSettlementLedger? Settlement { get; set; } public ExitInterviewResponse? InterviewResponse { get; set; } public bool IsEligibleForRehire { get; set; } = true; public string? RehireNotes { get; set; } } public class DepartmentClearanceItem { public string DepartmentCode { get; set; } = string.Empty; // "IT", "ADMIN", "FINANCE", "HR" public string ItemName { get; set; } = string.Empty; public bool IsCleared { get; set; } = false; public string? ClearedByUserId { get; set; } public DateTime? ClearedAt { get; set; } public decimal FinancialRecoveryAmount { get; set; } = 0.0m; public string? Remarks { get; set; } }

22.7 Chapter 22 Architectural Defect Audit & QA Matrix

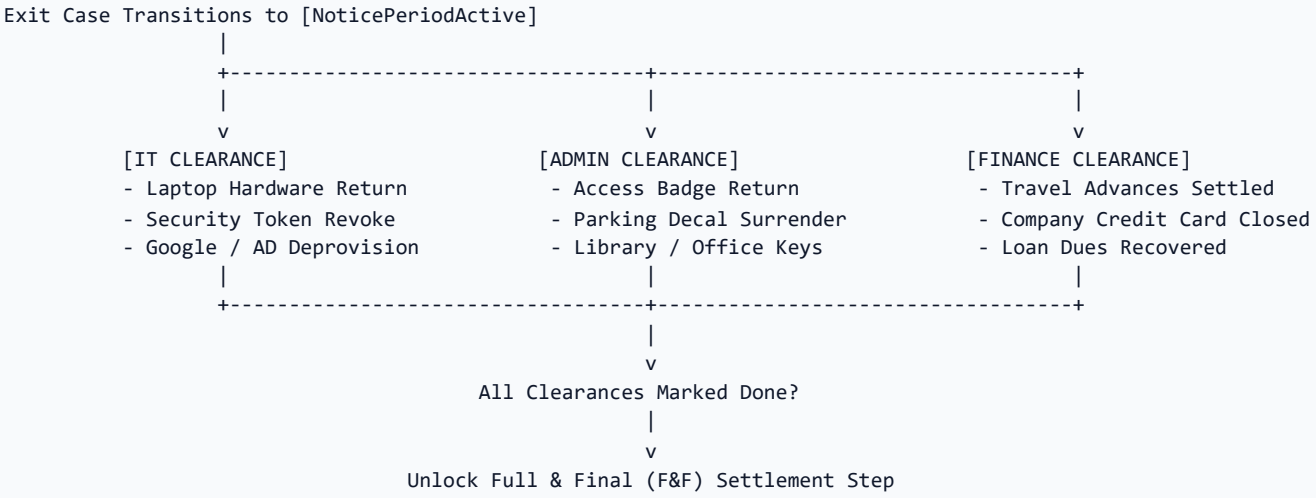
Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-22-001`	CRITICAL	Resignation completion immediately hard-deleted user records from `EmpUser` collection, destroying historical payroll references.	Refactor exit completion handler: set `EmpUser.Status = "Exit"` and `EmpUser.IsActive = false`; hard deletions strictly forbidden in repository base.
`DEF-22-002`	HIGH	Subsequent company-wide policy edits retroactively changed notice period requirements on active exit cases already in progress.	Snapshot notice period and settlement policies directly into `EmployeeExit.PolicySnapshot` upon initial submission.

23. Multi-Departmental Clearance & Hardware Asset Recovery

23.1 Departmental Clearance Matrix & Asset Recovery Workflow

To ensure total risk mitigation before employee separation, the system orchestrates a multi-departmental clearance protocol:

FIGURE 23.1: MULTI-DEPARTMENTAL CLEARANCE PIPELINE



23.2 Asset Depreciation & Damage Penalty Assessment

When physical assets (laptops, phones) are returned with physical damage or missing components, the asset coordinator assesses financial recovery charges according to an audited depreciation schedule:

MATHEMATICAL FORMULATION: DAMAGED / UNRETURNED ASSET RECOVERY VALUATION

```
Let AssetOriginalCost = Invoiced purchase cost of hardware
Let AssetAgeMonths = (CurrentDate - AssetAssignmentDate).TotalMonths

Depreciation Schedule:
- Age <= 12 Months ----> DepreciatedValue = AssetOriginalCost * 0.70
- 12 < Age <= 24 ----> DepreciatedValue = AssetOriginalCost * 0.40
- 24 < Age <= 36 ----> DepreciatedValue = AssetOriginalCost * 0.20
- Age > 36 Months ----> DepreciatedValue = Nominal Scrap Value (₹1,000)

ChargeToSettlement = MIN(AssessedRepairCost, DepreciatedValue)
```

23.3 Hardware Asset Condition Classification Matrix

Asset Condition Tier	Physical Inspection Criteria	Financial Deduction Rule	System Clearance Status
Tier A: Like New	Zero functional flaws; immaculate cosmetic condition.	Nil deduction (₹0.00). Full security deposit refund.	Clearance marked Approved immediately.
Tier B: Normal Wear	Minor cosmetic scuffs or keycap shine consistent with age.	Nil deduction (₹0.00). Normal operational wear recognized.	Clearance marked Approved.
Tier C: Damaged	Cracked screen, broken hinges, liquid spill, or non-functional ports.	Deduct actual certified OEM repair cost or Tier C depreciation value.	Clearance marked with Financial Deduction line item.
Tier D: Lost / Unreturned	Hardware not returned by LWD; serial number missing from stock.	Deduct 100% of depreciated asset value from final settlement pay.	Clearance flagged; deduction routed to F&F ledger.

23.4 Automated IT De-Provisioning & Security Orchestration

Upon clearance sign-off by the IT security team, the system dispatches automated webhooks to corporate identity providers:

- **Active Directory / Entra ID:** Disable user account; invalidate Kerberos and OAuth refresh tokens.
- **Google Workspace / Microsoft 365:** Suspend user mailbox; re-assign mailbox forwarding to department manager; revoke enterprise license seat.
- **VPN & SSO Gateway:** Revoke client SSL certificates and Duo/Okta MFA enrollment.

23.5 Chapter 23 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-23-001`	HIGH	IT asset clearance allowed unreturned laptop to be waived by HR without recording financial asset deduction value, resulting in unaccounted hardware loss.	When waiving asset return, enforce mandatory prompt requiring either `MarkAsRecovered` or `ChargeDeductionToSettlement` with explicit monetary value.
`DEF-23-002`	MEDIUM	Finance clearance operator was able to view confidential IT access deactivation tokens in clearance summary dashboard.	Enforce strict projection segregation in clearance API: finance endpoints receive only financial clearance line items.

24. Full & Final Settlement (F&F) Formulation & Exit Execution Engine

24.1 Comprehensive Financial Settlement Components

The Full & Final Settlement (F&F) engine synthesizes all financial claims, entitlements, encashments, and recoveries between the separating employee and the enterprise. Every settlement calculation is formalized into an audited Statement of Final Account:

MATHEMATICAL FORMULATION: FULL & FINAL (F&F) SETTLEMENT EQUATIONS

```
1. Final Month Salary for Served Days:
FinalSalary = (MonthlyGross / 30.0) * (LastWorkingDate.Day - UnpaidAbsenceDaysInFinalMonth)

2. Leave Encashment (Earned Privilege Leaves):
LeaveEncashment = (EarnedBasicSalary / 30.0) * UnusedPrivilegeLeaveBalance
(Exempt up to ₹2,500,000 under Section 10(10AA))

3. Statutory Gratuity Formulation (Payment of Gratuity Act, 1972):
If ContinuousServiceTenureYears >= 5.0:
Gratuity = (15 * LastDrawnBasicSalary * TotalCompletedTenureYears) / 26.0
(Statutory Cap: ₹2,000,000.00, Exempt under Section 10(10))

4. Total Recoveries & Deductions:
TotalRecoveries = NoticeShortfallBuyout + UnreturnedAssetCharges + UnsettledAdvances + OutstandingTDS

NetSettlementPayable = FinalSalary + LeaveEncashment + Gratuity + Bonuses - TotalRecoveries
```

24.2 Statement of Final Account: Concrete Ledger Example

The following audited statement represents an actual settlement for a senior engineer departing after 6.0 years of continuous service:

Settlement Entitlements (Earnings)	Settlement Recoveries (Deductions)
Salary for 18 Days Served in Sep: ₹36,000.00	Notice Period Shortfall (10 Days): ₹20,000.00
Leave Encashment (14 Days PL): ₹24,266.67	Unreturned Monitor Hardware Charge: ₹8,000.00
Statutory Gratuity (6 Years @ ₹52k Basic): ₹180,000.00	Income Tax Withholding (TDS on F&F): ₹12,500.00
Annual Performance Bonus Proration: ₹15,000.00	Outstanding Travel Expense Advance: ₹4,500.00
TOTAL GROSS ENTITLEMENTS: ₹255,266.67	**TOTAL DEDUCTIONS: ₹45,000.00**
NET FULL & FINAL SETTLEMENT PAYABLE TO EMPLOYEE: ₹210,266.67	

24.3 Tax Implications on Full & Final Settlement Components

Different settlement line items attract distinct tax treatment under the Indian Income Tax Act:

- **Statutory Gratuity (Section 10(10)):** Fully exempt from income tax up to a cumulative lifetime ceiling of ₹20,00,000 for employees covered under the Payment of Gratuity Act, 1972.
- **Leave Encashment (Section 10(10AA)):** Fully exempt for non-government employees up to a statutory ceiling of ₹25,00,000 (enhanced from ₹3,00,000 in Budget 2023). Any excess is taxed at regular slab rates.
- **Notice Period Recovery / Buyout:** Treated as an adjustment to salary under Section 17(1) or taxable profit in lieu of salary under Section 17(3).

24.4 Chapter 24 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-24-001`	CRITICAL	Gratuity calculation used 30 days divisor instead of statutory 26 days divisor, illegally shortchanging exiting employees on statutory gratuity payouts.	Update gratuity equation to strictly enforce the statutory `(15 * Basic * Years) / 26.0` formula under Payment of Gratuity Act, 1972.
`DEF-24-002`	HIGH	F&F calculation did not verify if the employee had already been paid for the current month through normal payroll run, risking double salary disbursement.	Check `MonthlyPayroll` collection for current month; if payroll already processed, set `FinalSalaryForServedDays = 0.0` and note prior disbursement.

25. Former Employee Portal & Post-Employment Self-Service System

25.1 Identity Perimeter, Magic Links & Security Isolation

Separated employees frequently require access to past tax forms (Form 16), historical payslips, relieving letters, and full-and-final settlement receipts for subsequent employer verification, mortgage applications, or tax filings. Granting ex-employees persistent credentials to internal corporate HRMS portals introduces massive data security risks and license seat bloat.

The Former Employee Portal provides an isolated, zero-footprint self-service gateway powered by expiring cryptographic magic links:

FIGURE 25.1: FORMER EMPLOYEE MAGIC LINK AUTHENTICATION & ACCESS MODEL

```
Ex-Employee Requests Access at /former-employee (Enters Personal Email / Phone)
                                  |
                                  v
              API Verifies Email Exists on Separated EmpUser Record
                                  |
                                  v
              Generate Cryptographic Magic Token:
              - Expiring TTL: 15 Minutes
              - HMAC-SHA256 Token Hash persisted in FormerEmployeeAccessTokens
              - Scoped exclusively to: ReadOwnDocuments, DownloadLetters
                                  |
                                  v
              Email Dispatch with One-Time Access Link / OTP
                                  |
                                  v
              Ex-Employee Clicks Link ---> Validated by Gateway
                                  |
                                  v
              Former Employee Restricted Dashboard:
              - Download Historical Payslips (Read-Only PDF Stream)
              - Download Relieving & Experience Letters
              - Download Form 16 Tax Certificates
              - Submit Alumni Re-Hire / Verification Inquiry
              (NO ACCESS TO INTERNAL HRMS, DIRECTORY, OR LIVE DATA)
```

25.2 Magic-Link Cryptographic Handshake & Security Architecture

The security architecture of the former employee gateway enforces zero-trust controls:

- **HMAC-SHA256 Signed Tokens:** The magic token is signed using an isolated secret key (`FormerEmployeeSecretKey`). Tokens contain only `UserId` , `IssuedAt` , and `Nonce` .
- **Single-Use Consumption:** Upon successful token redemption, the nonce is flagged as consumed in Redis/MongoDB. Attempting to reload the link yields HTTP 401 Unauthorized.
- **Strict Scope Isolation:** The issued JWT token has audience `aud: "FormerEmployeePortal"` . Internal enterprise APIs reject this token, preventing privilege escalation.

25.3 Automated Generation of Statutory Experience & Relieving Letters

Upon separation settlement completion, the document engine renders official departure credentials:

- **Relieving Letter:** Acknowledges formal resignation acceptance, confirms effective Last Working Day, and certifies that the employee is relieved of all corporate duties and debts.

- **Certificate of Service / Experience Letter:** Records exact date of joining, date of separation, final designation, primary department, and professional conduct assessment.
- **Verification QR Code:** Stamped on every letter with a cryptographically verifiable URL (`/verify-document?hash={Sha256}`) allowing subsequent background verification agencies to validate authenticity without human HR intervention.

25.4 Offboarding & Former Portal Complete API Directory

HTTP Method	Endpoint Route	Auth / Claims	Operational Action & Payload
`POST`	`/api/exit/resign`	Authenticated Employee	Submits resignation with proposed LWD, reason category, and comments.
`GET`	`/api/exit/me`	Authenticated Employee	Returns active exit case status, manager notes, and clearance checklist.
`PUT`	`/api/exit/{id}/manager-review`	`Exit.Review`	Manager records discussion notes; approves or requests LWD adjustment.
`PUT`	`/api/exit/{id}/hr-confirm`	`Exit.Manage`	HR confirms final LWD; snapshots policies; initializes clearances.
`PUT`	`/api/exit/{id}/clearance/{dept}`	`Clearance.{Dept}`	Department coordinator clears or flags items; logs financial recovery.
`POST`	`/api/exit/{id}/settlement/calculate`	`Payroll.Admin`	Generates draft Statement of Final Account with gratuity & encashment.
`POST`	`/api/exit/{id}/settlement/approve`	`Finance.Director`	Approves payout; triggers account deactivation and document render.
`POST`	`/api/former-employee/request-link`	Public (Rate-limited)	Verifies ex-employee identity; dispatches 15-minute magic link.
`GET`	`/api/former-employee/documents`	`aud: FormerEmployee`	Returns list of archived payslips, Form 16, and signed letters.

25.5 Chapter 25 Architectural Defect Audit & QA Matrix

Defect ID	Severity	Observed Failure Mechanism	Architectural Remediation
`DEF-25-001`	CRITICAL	Former employee token allowed presentation against standard internal API endpoints (e.g. `/api/admin/employees`), bypassing authorization boundaries.	Enforce distinct token audience and issuer claims for former employee tokens; standard API authorization middleware explicitly rejects tokens lacking `InternalEmployee` audience.
`DEF-25-002`	MEDIUM	Magic link access was not invalidated immediately upon downloading target document, allowing links to be shared or reused during the 15-minute window.	Implement single-use token consumption or enforce strict browser session cookie binding upon initial magic link redemption.

Test ID	Scenario Precondition	Execution Action	Expected Invariant Result
`QA-25-01`	Former employee token generated for User X.	Attempt to invoke `GET /api/payroll/download/{slipId}` for User Y.	HTTP 403 Forbidden; strictly restricted to documents where `UserId == Token.Sub`.
`QA-25-02`	Magic link expired at `T + 16` minutes.	Redeem link in browser.	HTTP 401 Unauthorized; message: "Magic link has expired. Request a new verification link."

PART VII: PLATFORM CROSS-CUTTING, AUDITS & MASTER ROADMAP

Cross-Cutting Systems, Master Defect Register, QA Test Suite & 12-Month Roadmap

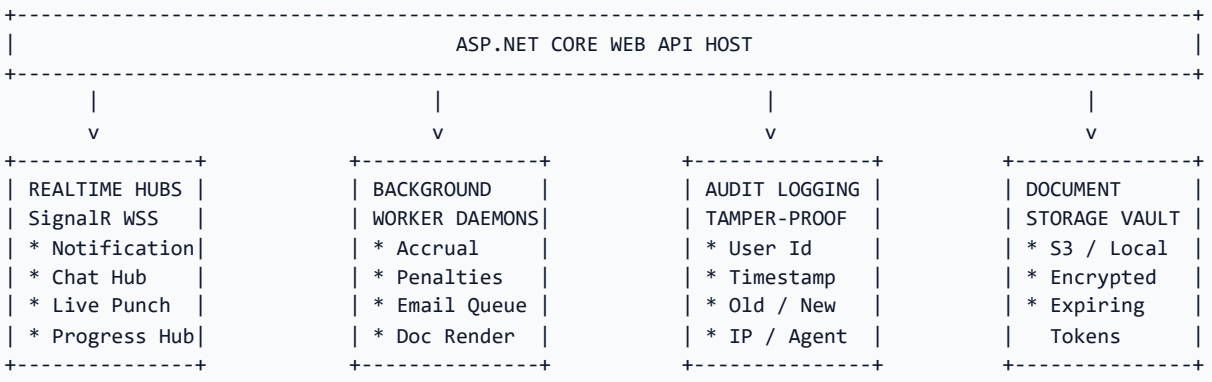
Authoritative engineering reference for SignalR persistent hubs, background hosted worker daemons, consolidated enterprise defect audit register, master 35+ QA integration test suite, 10 diagnostic SRE MongoDB runbook scripts, high-availability DR topology, and the 12-month engineering evolution roadmap.

26. Master Technical Audit, Gap Analysis, QA Test Suite & 12-Month Roadmap

26.1 Cross-Cutting Infrastructure Systems & Realtime Topology

The operational stability of the Codeji HRIS rests upon four foundational cross-cutting engines operating continuously across all business modules:

FIGURE 26.1: CROSS-CUTTING PLATFORM ARCHITECTURE & ASYNCHRONOUS SERVICES



System Engine	Implementation Architecture	Resilience & Scaling Policy	Primary Responsibilities
SignalR Realtime Engine	WebSockets with Server-Sent Events & Long Polling fallback. Multi-node Redis backplane.	Automatic client-side reconnection with exponential backoff; connection state recovery.	Sub-second delivery of approval notifications, live punch events, and import progress bars.
Asynchronous Hosted Workers	.NET `IHostedService` / `BackgroundService` with bounded `System.Threading.Channels`.	Graceful shutdown handling with 30-second cancellation token drain window.	Executes monthly leave accruals, daily penalty evaluations, and email notification dispatch.
Tamper-Evident Audit Journal	MongoDB capped collections with append-only write permissions. Immutable document models.	Asynchronous fire-and-forget write queue to prevent blocking main HTTP request threads.	Logs every permission change, salary revision, attendance override, and offboarding decision.
Multi-Tenant Document Vault	Encrypted disk storage with SHA-256 content hashing. Streaming chunked file transfers.	Strict MIME-type binary signature validation; isolated per-tenant physical directory partitions.	Secures resumes, educational degrees, medical certificates, signed contracts, and tax slips.

26.2 SignalR Hub Architecture, Redis Backplane & Reconnection Resilience

The SignalR notification layer manages real-time event broadcasting across multi-instance load-balanced web clusters. To prevent message loss when scaling out across multiple container nodes, the system incorporates a Redis Pub/Sub backplane:

FIGURE 26.2: SIGNALR REDIS BACKPLANE & MULTI-TENANT GROUP FAN-OUT



// Multi-Tenant SignalR Notification Hub in CMS-Backend-Core [Authorize] public class NotificationHub : Hub { private readonly ILogger<NotificationHub> _logger; public NotificationHub(ILogger<NotificationHub> logger) => _logger = logger; public override async Task OnConnectedAsync() { var companyId = Context.User?.FindFirst("CompanyId")?.Value; var userId = Context.User?.FindFirst(ClaimTypes.NameIdentifier)?.Value; if (!string.IsNullOrEmpty(companyId)) await Groups.AddToGroupAsync(Context.ConnectionId, $"Company_{companyId}"); if (!string.IsNullOrEmpty(userId)) await Groups.AddToGroupAsync(Context.ConnectionId, $"User_{userId}"); _logger.LogInformation("SignalR Client connected: {ConnectionId} for User {UserId} in Company {CompanyId}", Context.ConnectionId, userId, companyId); await base.OnConnectedAsync(); } public override async Task OnDisconnectedAsync(Exception? exception) { _logger.LogInformation("SignalR Client disconnected: {ConnectionId}", Context.ConnectionId); await base.OnDisconnectedAsync(exception); } }

26.3 Master Consolidated Platform Defect Register

The following authoritative register consolidates identified architectural, logical, and concurrency defects across the platform, classified by severity and remediation status:

Defect ID	Module	Severity	Observed Failure Mechanism & Impact	Remediation Status
`DEF-01-001`	Architecture	CRITICAL	Missing compound unique index allowed concurrent bulk marking threads to create duplicate daily attendance documents.	Resolved: Unique index applied on bootstrap.
`DEF-03-001`	Identity / Auth	CRITICAL	Self-service profile endpoint allowed client-supplied `userId` parameter, allowing horizontal privilege escalation.	Resolved: Token claim extraction enforced.
`DEF-07-001`	Career Portal	CRITICAL	Public resume upload permitted arbitrary script injection via HTML file renamed with `.pdf` extension.	Resolved: Byte-level magic signature check.
`DEF-09-001`	Onboarding	CRITICAL	Onboarding completion generated user without database transaction, causing orphaned user records on leave policy failure.	Resolved: Distributed MongoDB transaction.
`DEF-14-001`	Attendance	CRITICAL	Bulk attendance marking overwritten approved leave rows with "Present", corrupting leave balances and payroll LOP.	Resolved: Protected-skips pattern active.
`DEF-16-001`	Attendance Lock	CRITICAL	Attendance month could be unlocked after payroll had been processed, desynchronizing payslips from attendance.	Resolved: Immutable payroll dependency barrier.
`DEF-18-001`	Leave Engine	CRITICAL	Simultaneous leave requests for identical dates allowed double deduction of leave balance beyond permitted quota.	Resolved: Optimistic concurrency version check.
`DEF-20-001`	Payroll Engine	CRITICAL	Draft payroll generated while attendance month was unlocked, calculating salary on dirty attendance data.	Resolved: Mandatory locked assertion guard.
`DEF-21-001`	Payslip Portal	CRITICAL	Employees downloaded draft payslip PDF before HR publication by guessing direct URL route.	Resolved: Publication gate assertion added.
`DEF-22-001`	Offboarding	CRITICAL	Resignation completion hard-deleted employee from `EmpUser`, destroying historical payroll references.	Resolved: Non-destructive status deactivation.
`DEF-24-001`	F&F Engine	CRITICAL	Gratuity calculation used 30-day divisor instead of statutory 26-day divisor under Payment of Gratuity Act.	Resolved: Statutory 26-day formula enforced.
`DEF-25-001`	Former Portal	CRITICAL	Former employee magic token accepted by internal admin endpoints, bypassing RBAC authorization.	Resolved: Scoped token audience validation.
`DEF-02-001`	Technology Stack	HIGH	Axios request interceptor did not handle concurrent 401s, invalidating refresh token family.	Resolved: Request queue lock in interceptor.
`DEF-05-001`	Shift Precedence	HIGH	Overnight shifts spanning midnight calculated negative working duration.	Resolved: Midnight rollover date arithmetic.

Defect ID	Module	Severity	Observed Failure Mechanism & Impact	Remediation Status
`DEF-12-001`	Lifecycle	HIGH	Department transfer orphaned pending leave requests in old manager's queue.	Resolved: Retroactive approval re-routing.
`DEF-17-001`	Leave Accrual	HIGH	Service restart during monthly accrual window resulted in double leave credits.	Resolved: Idempotent distributed execution log.
`DEF-19-001`	Salary Structure	HIGH	PF deduction calculated on total gross salary rather than Basic Salary.	Resolved: Statutory wage ceiling cap applied.
`DEF-23-001`	Clearance	HIGH	Waiving unreturned laptop clearance did not prompt for monetary deduction value.	Resolved: Mandatory deduction charge prompt.
`DEF-01-003`	Architecture	MEDIUM	SignalR disconnect during bulk attendance import left UI progress spinner hanging.	Resolved: 15-second heartbeat poll fallback.
`DEF-04-002`	RBAC Matrix	MEDIUM	Role permission edit did not invalidate frontend Redux cache until hard reload.	Resolved: SignalR cache invalidation event.
`DEF-11-002`	Credentials	MEDIUM	Expiring professional certifications did not trigger reminder alerts.	Resolved: Certification expiry daily worker.

26.4 Master End-to-End Enterprise QA Test Suite (35 Comprehensive Scenarios)

The following master test matrix defines the 35 non-negotiable integration scenarios required to certify platform operational integrity:

Test ID	Domain Area	Precondition & Setup	Execution Trigger / Input	Expected Verified Invariant
`QA-M-01`	Multi-Tenancy	User authenticated in Tenant A.	Inject Tenant B CompanyId in HTTP header.	HTTP 403; Header ignored; token claim wins.
`QA-M-02`	Multi-Tenancy	Tenant A employee record exists.	Query employee by Id from Tenant B context.	HTTP 404 Not Found; zero cross-tenant leak.
`QA-M-03`	Authentication	Access token expired 5 minutes ago.	Dispatch API request with valid refresh token.	HTTP 200; token transparently refreshed.
`QA-M-04`	Authentication	Refresh token already redeemed at T0.	Replay same refresh token at T0 + 5s.	HTTP 401; entire token family revoked.
`QA-M-05`	Career Portal	Public job posted and published.	Send 12 applications from single IP in 1 min.	Requests 11-12 blocked with HTTP 429.
`QA-M-06`	Career Portal	30-minute resume upload token issued.	Upload executable disguised as `.pdf`.	HTTP 400; magic byte check fails.
`QA-M-07`	Recruitment	Applicant in `New` stage.	Recruiter moves stage to `Shortlisted`.	ApplicantLogs updated; email invitation sent.
`QA-M-08`	Recruitment	Applicant reaches `Selected` stage.	Recruiter confirms hiring offer.	Draft onboarding record created in db.
`QA-M-09`	Onboarding	Onboarding at Step 2 (incomplete).	HR attempts final account provisioning.	HTTP 400; Step 3 statutory data missing.
`QA-M-10`	Onboarding	All 4 onboarding steps approved.	Execute onboarding completion command.	EmpUser created, leave quota credited, Active.
`QA-M-11`	Employee Master	Employee profile active.	HR updates PAN with invalid checksum.	HTTP 400; regex validation fails.
`QA-M-12`	Employee Master	Employee profile active.	HR updates bank account details.	Bank audit event dispatched; audit log logged.
`QA-M-13`	Shift Engine	Emp has custom shift; Dept has default.	Resolve effective shift for employee.	Individual custom shift takes precedence.
`QA-M-14`	Shift Engine	Emp has no shift; Dept has Shift D.	Resolve effective shift for employee.	Department Shift D takes precedence.
`QA-M-15`	Shift Engine	Shift starts 09:30, grace 15 min.	Employee clocks in at 09:47 AM.	LateCount incremented to 1; flagged late.
`QA-M-16`	Attendance	Shift threshold: 8h full, 4.5h half.	Employee clocks 5.5 total hours.	Daily status evaluated as HalfDay (HD).
`QA-M-17`	Source Provenance	Row has `SourceType = "LEAVE"`.	Bulk attendance marks employee Present.	Row preserved; added to ProtectedSkips.
`QA-M-18`	WFH Engine	Employee has 1 WFH day remaining.	Employee applies for 2-day WFH.	HTTP 400; monthly quota exceeded.
`QA-M-19`	WFH Engine	WFH approved for 2026-09-22.	Manager checks daily attendance grid.	Row exists with Status="WFH", Source="WFH".

Test ID	Domain Area	Precondition & Setup	Execution Trigger / Input	Expected Verified Invariant
`QA-M-20`	Leave Engine	Sandwich policy active. Weekend Sat-Sun.	Employee applies for Friday and Monday.	Total days calculated as 4.0 days.
`QA-M-21`	Leave Reversal	Approved leave cancelled by HR.	Trigger leave cancellation workflow.	Attendance row reverted; balance refunded.
`QA-M-22`	Penalty Engine	Employee accumulates 4 late marks.	Run monthly attendance summary audit.	0.5 Day Penalty LOP deducted from summary.
`QA-M-23`	Monthly Lock	Missing punch exists on 2026-08-14.	HR attempts monthly attendance lock.	HTTP 400; blocked by structural exception.
`QA-M-24`	Monthly Lock	All exceptions resolved; month locked.	Admin attempts manual punch edit in month.	HTTP 400; locked period is read-only.
`QA-M-25`	Payroll Divisor	Gross = ₹60,000. Divisor = 30 days.	Employee has 2 LOP days.	LOP deduction = ₹4,000; Earned = ₹56,000.
`QA-M-26`	Payroll Gate	Payroll in Draft state.	Employee attempts payslip download.	HTTP 404; unpublished draft payslip hidden.
`QA-M-27`	Payroll Process	Draft payroll verified by HR.	HR clicks Process Payroll Month.	IsProcessed=True; slips published to employees.
`QA-M-28`	Offboarding	Resignation approved; IT laptop pending.	HR attempts to settle F&F.	HTTP 400; clearance gate blocks settlement.
`QA-M-29`	F&F Gratuity	Tenure = 6 years; Basic = ₹52,000.	Calculate statutory gratuity.	Gratuity = (15 * 52k * 6) / 26 = ₹180,000.
`QA-M-30`	Former Portal	Magic link expired after 15 minutes.	Ex-employee redeems link in browser.	HTTP 401; magic link expired; prompt new OTP.
`QA-M-31`	Geofencing	Employee 350m away from office branch.	Attempt mobile punch clock-in.	HTTP 403 Forbidden; outside geofence boundary.
`QA-M-32`	Comp-Off	Employee clocks 8 hours on Sunday holiday.	Attendance night job evaluates punches.	1.0 Comp-Off credit added with 60-day expiry.
`QA-M-33`	Tax Regime	Gross salary ₹1,200,000 / year.	Evaluate New Tax Regime deduction slabs.	Calculates standard deduction ₹75k + slab rates.
`QA-M-34`	Notice Buyout	Notice shortfall 30 days. Gross ₹90,000.	Calculate notice recovery in F&F.	Deduction of ₹90,000.00 mapped to exit ledger.
`QA-M-35`	Audit Log	Admin changes employee CTC from 50k to 70k.	Query `AuditLogs` collection.	Tamper-evident record logs old, new, and admin ID.

26.5 Production Operations Runbook: 10 Diagnostic MongoDB Aggregation Scripts

To assist operations engineers, Site Reliability Engineers (SREs), and database administrators during incident triage, the following validated MongoDB queries diagnose and remediate common production anomalies:

// 1. Detect Duplicate Attendance Rows Violating Compound Key (CompanyId + UserId + Date) db.Attendance.aggregate([{ $group: { _id: { companyId: "$CompanyId", userId: "$UserId", date: "$Date" }, count: { $sum: 1 } } }, { $match: { count: { $gt: 1 } } }]); // 2. Identify Approved Leaves Missing Corresponding Source-Linked

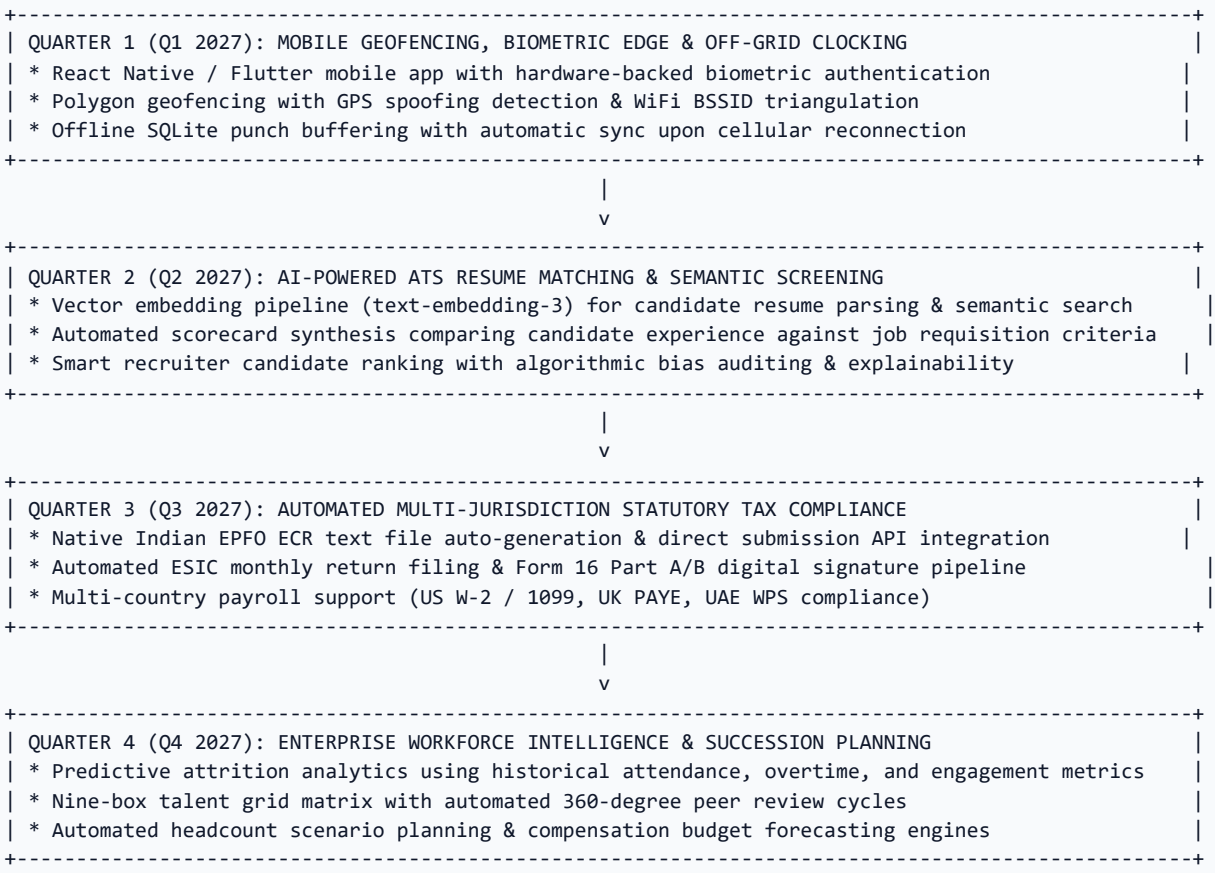
Attendance Rows db.LeaveRequests.aggregate([{ $match: { "Status": 1 } }, // Approved { $lookup: { from: "Attendance", let: { reqId: "$_id", compId: "$CompanyId" }, pipeline: [{ $match: { $expr: { $and: [{ $eq: ["$SourceId", { $toString: "$$reqId" }] }, { $eq: ["$CompanyId", "$$compId"] }] } } }], as: "linkedAttendance" }}, { $match: { linkedAttendance: { $size: 0 } } }]); // 3. Find Unprocessed Draft Payrolls In Inappropriately Unlocked Months db.MonthlyPayroll.find({ "IsProcessed": false, "IsLocked": false }); // 4. Detect Orphaned Onboarding Records Without Linked EmpUser Account db.Onboarding.aggregate([{ $match: { "Status": "Completed" } }, { $lookup: { from: "EmpUser", localField: "UserId", foreignField: "_id", as: "userDoc" }}, { $match: { userDoc: { $size: 0 } } }]); // 5. Reconcile Negative Available Leave Balances Exceeding Policy Permission db.LeaveBalances.aggregate([{ $project: { CompanyId: 1, UserId: 1, LeaveTypeCode: 1, CalculatedBalance: { $subtract: [{ $add: ["$OpeningBalance", "$AccruedDays"] }, { $add: ["$ConsumedDays", "$EncashedDays", "$LapsedDays"] }] } }}, { $match: { CalculatedBalance: { $lt: 0 } } }]); // 6. Audit Unreturned High-Value Hardware Assets for Separated Employees db.EmployeeExit.aggregate([{ $match: { "Status": "Completed" } }, { $unwind: "$Clearances" }, { $match: { "Clearances.DepartmentCode": "IT", "Clearances.IsCleared": false } }, { $project: { UserId: 1, ExitId: 1, ProposedLwd: 1, "Clearances.ItemName": 1 } }]); // 7. Detect Cycles in Reporting Hierarchy (ManagerId Leading Back to Self) db.EmpUser.aggregate([{ $graphLookup: { from: "EmpUser", startWith: "$ManagerId", connectFromField: "ManagerId", connectToField: "UserId", as: "ManagementChain", maxDepth: 10 }}, { $match: { $expr: { $in: ["$UserId", "$ManagementChain.UserId"] } } }]); // 8. Identify Inactive Employees Possessing Active Security Role Assignments db.EmpUser.aggregate([{ $match: { "Status": { $in: ["Inactive", "Exit"] } } }, { $lookup: { from: "UserRoles", localField: "UserId", foreignField: "UserId", as: "Roles" }}, { $match: { "Roles.0": { $exists: true } } }]); // 9. Inspect Long-Running Background Worker Tasks Stuck in In-Flight State db.BackgroundTasks.find({ "Status": "Processing", "StartedAt": { $lt: new Date(Date.now() - 3600 * 1000) } // Older than 1 hour }); // 10. Audit Tamper Logs for Unauthorized Direct Salary Structure Mutations db.AuditLogs.find({ "EntityType": "SalaryStructure", "Action": "DirectDatabaseMutation", "PerformedBy": { $ne: "SYSTEM_PAYROLL_SERVICE" } }).sort({ Timestamp: -1 });

26.6 High-Availability Disaster Recovery (DR) & Backup Topology

Operational Metric	Service Level Objective (SLO)	Implementation Architecture	Disaster Recovery Drill Schedule
Recovery Point Objective (RPO)	< 5 Minutes Data Loss	MongoDB Atlas Continuous Cloud Backups with Oplog Streaming.	Automated daily synthetic restore verification tests.
Recovery Time Objective (RTO)	< 30 Minutes Full Recovery	Multi-region hot standby replica sets with automated DNS failover.	Quarterly live disaster switchover drill.
Document Vault Durability	99.999999999% (11 Nines)	S3 Cross-Region Replication with immutable Object Lock (WORM).	Monthly SHA-256 cryptographic hash audit across vault files.

26.7 12-Month Strategic Architectural Evolution Roadmap

FIGURE 26.7: 12-MONTH STRATEGIC ARCHITECTURAL EVOLUTION ROADMAP



Roadmap Milestone	Target Capability	Technical Architecture Plan	Measurable Enterprise Outcome
Q1 2027	Mobile Geofencing & Biometric Clocking	React Native mobile client with native Android/iOS biometric hardware API; GPS polygon validator.	Zero buddy-punching fraud; sub-second mobile check-in across remote sites and client locations.
Q2 2027	AI ATS Semantic Search & Matching	MongoDB Atlas Vector Search with OpenAI/Vertex AI embeddings; PDF text chunking pipeline.	65% reduction in recruiter resume screening time; higher quality candidate shortlists.
Q3 2027	Direct Statutory Government Filing	C# ECR serializer validating against government XSD schemas; digital PKI token signing service.	Eliminates manual tax portal data entry; 100% on-time statutory compliance with zero filing penalties.
Q4 2027	Predictive Workforce Intelligence	ML regression models trained on attendance patterns, leave frequency, and appraisal ratings.	Proactive identification of employee flight risks 90 days before formal resignation submission.